



LLMs - Efficient LLMs

Benjamin Piwowarski

Lecture 3

<https://master-mind.isir.upmc.fr/llm/slides>

Press ? for help on keyboard shortcuts

[Slides in PDF](#)



Machine Learning &
Deep Learning for
Information Access

► WHY EFFICIENCY MATTERS?

MODERN LANGUAGE MODELS ARE POWERFUL BUT RESOURCE-INTENSIVE

- GPT-3 (175B): 650Gb memory
- LLaMA-2 (70B): 260Gb memory
- DistilBERT (66M): 251Mb memory

CHALLENGES

1. Training: Full fine-tuning is expensive (memory, compute, time)
 - Limits experimentation and iteration
 - Requires expensive GPUs
 - Slow to adapt to new tasks
2. Deployment: Large models don't fit on edge devices
 - Mobile phones have ~4-8GB RAM
 - Cloud inference costs scale with model size
3. Inference: Slow and costly at scale
 - User experience suffers from latency
 - Operating costs grow linearly with traffic
 - Energy consumption and environmental impact

Adaptation – Finetuning & Parameter-Efficient Methods

- Full finetuning vs. PEFT
- LoRA, Adapters, IA³, and other methods

Inference Optimization

- Quantization
- Flash Attention
- Speculative decoding

Efficient & Scalable Architectures

- Mixture of Experts
- Improving the attention mechanism
- Model compression

YOU HAVE TO SUBMIT A REPORT + CODE

🔑 Experimental study of PEFT

ADAPTATION: FINETUNING & PARAMETER-EFFICIENT METHODS

ADAPTATION: FINETUNING & PARAMETER-EFFICIENT METHODS

FULL MODEL FINETUNING

ADAPTATION: FINETUNING & PARAMETER-EFFICIENT METHODS ► WHY ADAPT PRE-TRAINED MODELS?

- Pre-trained LLMs have general language knowledge
(and partial task knowledge through instruction tuning)
- Task-specific adaptation improves performance

► Full Finetuning

Update all model parameters on task-specific data

- **Direct finetuning strategies**
 - Continue training with task-specific data
 - Use lower learning rate than pre-training

Full Finetuning

Update all model parameters on task-specific data

- **Direct finetuning strategies**
 - Continue training with task-specific data
 - Use lower learning rate than pre-training
- **Controlling finetuning**

Parameter freezing

keep some layers fixed

Layer-wise learning rates

different rates per layer

Gradual unfreezing

unfreeze layers progressively

⚠ Catastrophic Forgetting

Model loses pre-trained knowledge when finetuned on specific tasks

⚠ Catastrophic Forgetting

Model loses pre-trained knowledge when finetuned on specific tasks

- **Causes**

- Aggressive parameter updates
- Distribution shift between pre-training and finetuning
- Limited task-specific data

⚠ Catastrophic Forgetting

Model loses pre-trained knowledge when finetuned on specific tasks

- **Causes**

- Aggressive parameter updates
- Distribution shift between pre-training and finetuning
- Limited task-specific data

- **Mitigation strategies**

- Lower learning rates and careful scheduling
- Regularization (e.g. L2)
 - ⚠ L2 regularization should not be used on bias and normalization layers
- Replay methods: mix pre-training and task data
e.g. keeping MLM loss

► Learning Rate Scheduling

Adjust learning rate during training to improve convergence and prevent catastrophic forgetting

- Warmup prevents early instability
- Decay allows fine-grained optimization near convergence
- Typical max LR: $1e^{-5}$ to $5e^{-5}$ (vs $1e^{-4}$ for pre-training)

THERE ARE MANY TECHNIQUES

Linear warmup + decay

Gradually increase LR, then linearly decrease to 0

Cosine annealing

Smooth decay following cosine curve

Constant with warmup

Warmup then fixed LR (common in instruction tuning)

► Layer Freezing

Freeze (don't update) certain layers during finetuning, train only unfrozen layers

► Layer Freezing

Freeze (don't update) certain layers during finetuning, train only unfrozen layers

- **Basic approach**

- Freeze early layers (general features), train later layers (task-specific features)
- In NLP: freeze embedding + early transformer layers

Layer Freezing

Freeze (don't update) certain layers during finetuning, train only unfrozen layers

- **Basic approach**
 - Freeze early layers (general features), train later layers (task-specific features)
 - In NLP: freeze embedding + early transformer layers
- **Motivation**
 - Earlier layers learn general features, later layers learn specific features
 - Reduce compute and memory (no gradients for frozen layers)
 - Prevent catastrophic forgetting of general knowledge
 - Effective when limited task-specific data

Layer Freezing

Freeze (don't update) certain layers during finetuning, train only unfrozen layers

- **Basic approach**
 - Freeze early layers (general features), train later layers (task-specific features)
 - In NLP: freeze embedding + early transformer layers
- **Motivation**
 - Earlier layers learn general features, later layers learn specific features
 - Reduce compute and memory (no gradients for frozen layers)
 - Prevent catastrophic forgetting of general knowledge
 - Effective when limited task-specific data
- **Limitations**
 - Less flexible than full finetuning
 - Need to decide which layers to freeze (manual tuning)
 - May underperform full finetuning on complex tasks

1 Gradual Unfreezing

- Start with all layers frozen except last layer
- Train for N steps, then unfreeze next layer (last to first layers)

1 Gradual Unfreezing

- Start with all layers frozen except last layer
- Train for N steps, then unfreeze next layer (last to first layers)

2 Discriminative (layer-wise) learning rates

Lower LR for early layers, higher LR for later layers:

$$\eta^l = \eta^L \times \alpha^{L-l} \text{ with } \alpha \in [0.9, 0.95]$$

1 Gradual Unfreezing

- Start with all layers frozen except last layer
- Train for N steps, then unfreeze next layer (last to first layers)

2 Discriminative (layer-wise) learning rates

Lower LR for early layers, higher LR for later layers:

$$\eta^l = \eta^L \times \alpha^{L-l} \text{ with } \alpha \in [0.9, 0.95]$$

► ULMFit Combines the various above tricks

 *Universal Language Model Fine-tuning for Text Classification: 2018. <http://arxiv.org/abs/1801.06146>.*

1. Pre-train → freeze all → unfreeze last → unfreeze next → ... → finetune all
2. State-of-art for NLP before modern PEFT (2018)
3. Foundation for many transfer learning practices

ADAPTATION: FINETUNING & PARAMETER-EFFICIENT METHODS

PARAMETER-EFFICIENT FINETUNING (PEFT)

🎯 Goal

Adapt models while updating only a small fraction of parameters

🎯 Goal

Adapt models while updating only a small fraction of parameters

• Benefits

- Drastically reduced memory requirements
- Faster training
- Easy to store/deploy multiple task-specific models
- Often comparable performance to full finetuning

Goal

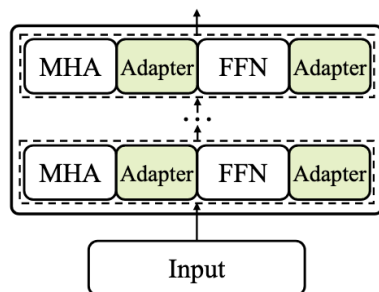
Adapt models while updating only a small fraction of parameters

Benefits

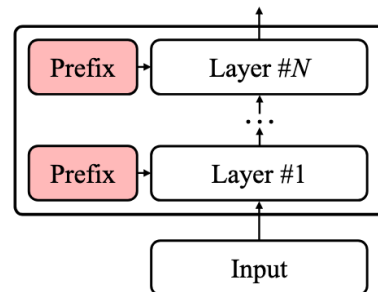
- Drastically reduced memory requirements
- Faster training
- Easy to store/deploy multiple task-specific models
- Often comparable performance to full finetuning

Key approaches

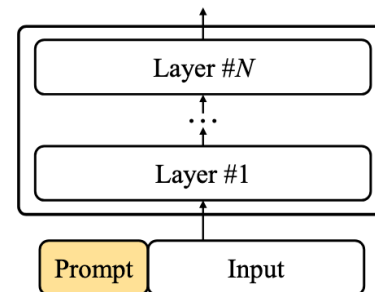
- Adapters (LoRA, Prefix tuning, IA³), Prompt tuning



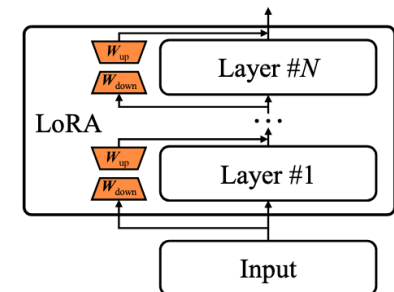
(a) Adapter Tuning



(b) Prefix Tuning



(c) Prompt Tuning



(d) Low-Rank Adaptation

▼ Adapters

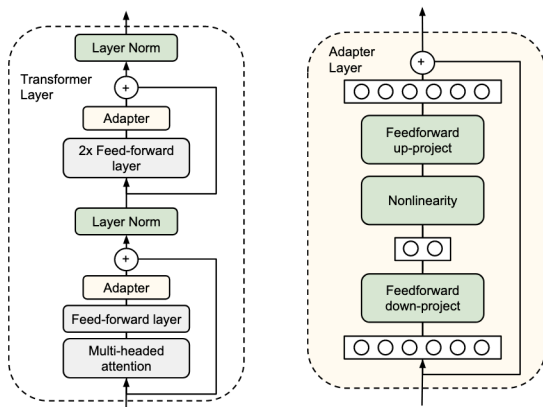
Small modules inserted into a transformer

Adapters

Small modules inserted into a transformer

Houlsby's adapters

 Houlsby, N. et al. 2019. *Parameter-Efficient Transfer Learning for NLP*.



Small bottleneck modules inserted between transformer layers

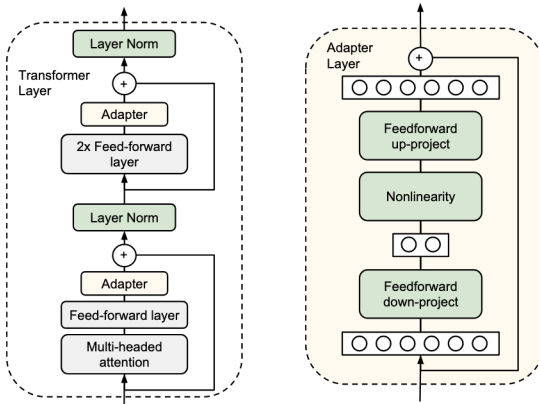
- Down-projection: $d \rightarrow r$ (bottleneck dimension)
- Non-linearity (e.g., ReLU)
- Up-projection: $r \rightarrow d$
- Residual connection

Adapters

Small modules inserted into a transformer

Houlsby's adapters

Houlsby, N. et al. 2019. *Parameter-Efficient Transfer Learning for NLP*.



Small bottleneck modules inserted between transformer layers

- Down-projection: $d \rightarrow r$ (bottleneck dimension)
- Non-linearity (e.g., ReLU)
- Up-projection: $r \rightarrow d$
- Residual connection

• Training

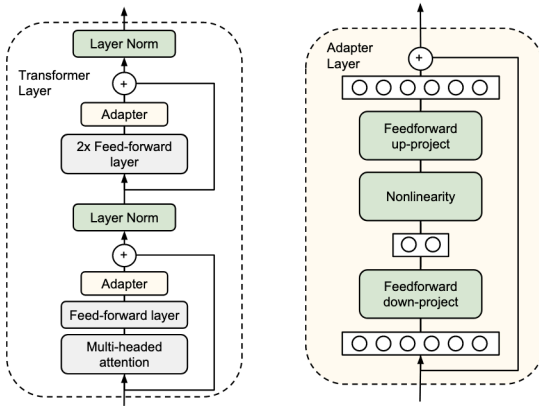
- Freeze transformer parameters
- Train only adapter parameters
- Typical: 1-5% of total parameters

Adapters

Small modules inserted into a transformer

Houlsby's adapters

Houlsby, N. et al. 2019. *Parameter-Efficient Transfer Learning for NLP*.



Small bottleneck modules inserted between transformer layers

- Down-projection: $d \rightarrow r$ (bottleneck dimension)
- Non-linearity (e.g., ReLU)
- Up-projection: $r \rightarrow d$
- Residual connection

• Training

- Freeze transformer parameters
- Train only adapter parameters
- Typical: 1-5% of total parameters

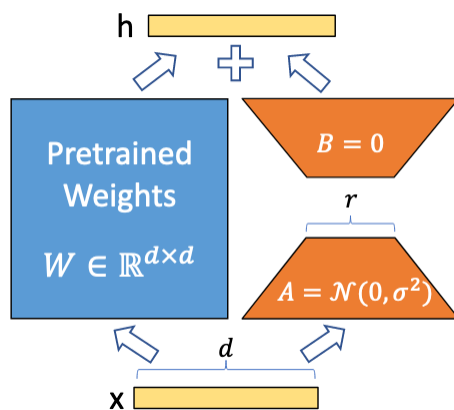
• Advantages

- Modular: swap adapters for different tasks
- Minimal inference overhead

ADAPTATION

 LoRA

📖 LoRA: Low-Rank Adaptation of Large Language Models: 2021. <http://arxiv.org/abs/2106.09685>.



🔗 **Hypothesis:** weight updates lie in low-rank subspace

🔗 Add learnable low-rank matrices to existing weights

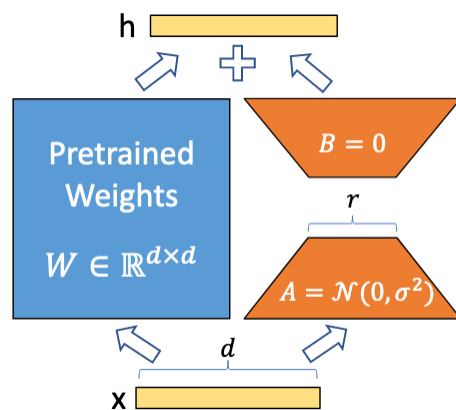
Represent weight updates as low-rank decomposition:

$$\Delta W = BA \text{ where } B \in \mathbb{R}^{d \times r}, A \in \mathbb{R}^{r \times k}, r \ll \min(d, k)$$

ADAPTATION

 LoRA

📖 LoRA: Low-Rank Adaptation of Large Language Models: 2021. <http://arxiv.org/abs/2106.09685>.



🔗 **Hypothesis:** weight updates lie in low-rank subspace

🔗 Add learnable low-rank matrices to existing weights

Represent weight updates as low-rank decomposition:

$$\Delta W = BA \text{ where } B \in \mathbb{R}^{d \times r}, A \in \mathbb{R}^{r \times k}, r \ll \min(d, k)$$

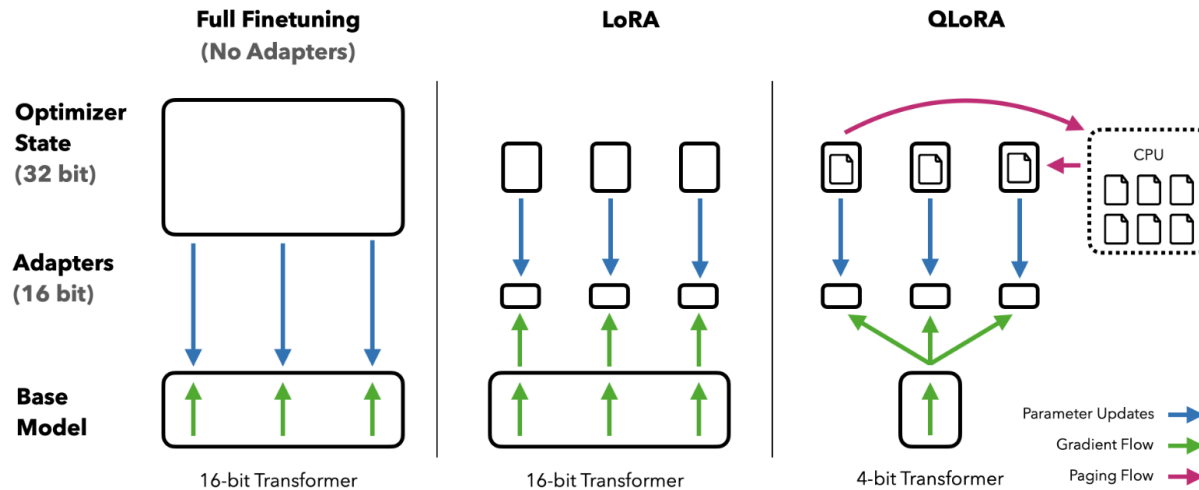
PROPERTIES

- Train only A and B (typically $r = 4$ to 64)
- No inference latency: merge $W_0 + BA$ after training
- Apply to attention weights (Q, K, V, O) or FFN

QLoRA

📖 QLoRA: Efficient Finetuning of Quantized LLMs: 2023. <http://arxiv.org/abs/2305.14314>.

combine with quantization (4-bit base model)



DoRA

📖 Liu, S.-Y. et al. 2024. DoRA: Weight-Decomposed Low-Rank Adaptation. *Proceedings of the 41st International Conference on Machine Learning*.

🔑 Decompose into magnitude and direction

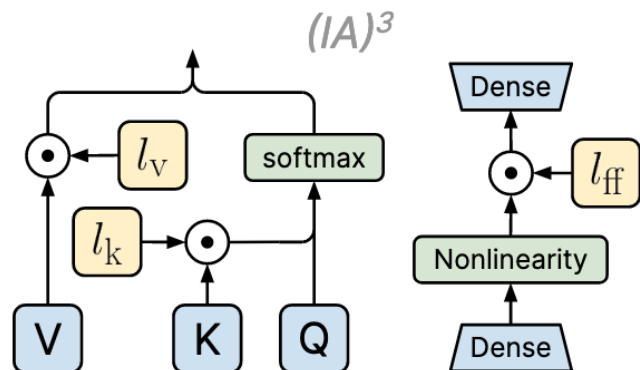
🔑 Uses LORA for learning the direction



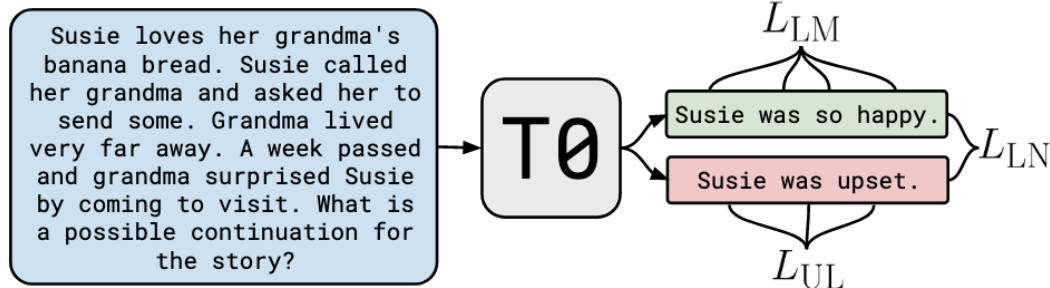
📖 Liu, H. et al. 2022. *Few-Shot Parameter-Efficient Fine-Tuning Is Better and Cheaper than In-Context Learning*. 36th Conference on Neural Information Processing Systems (NeurIPS 2022).

Learn element-wise scaling vectors that modulate activations

$h = l_k \odot W_k x$ (where $\odot$ is element-wise product)



Losses used in T-Few

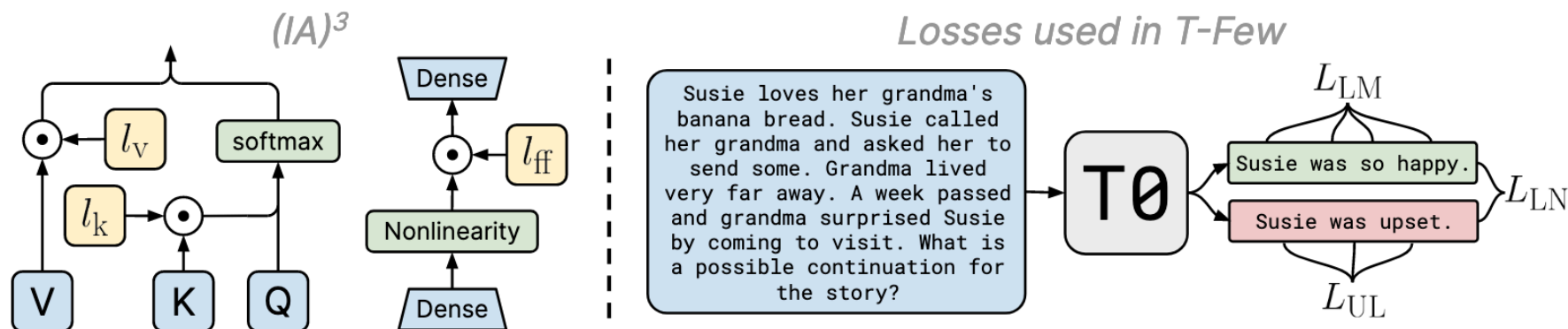




📖 Liu, H. et al. 2022. *Few-Shot Parameter-Efficient Fine-Tuning Is Better and Cheaper than In-Context Learning*. 36th Conference on Neural Information Processing Systems (NeurIPS 2022).

Learn element-wise scaling vectors that modulate activations

$h = l_k \odot W_k x$ (where $\odot$ is element-wise product)



- ✚ Even fewer parameters than LoRA (typically 0.01% of model)
- ✚ No added inference latency
- ✚ Rescaling is cheap computationally

- ✖ More constrained than LoRA
- ✖ May underperform on complex tasks

INSPIRATION: HARD PROMPTS

Direct model instruction

🔑 **Example:** "Translate to French:"

➕ interpretable

➖ not optimal

INSPIRATION: HARD PROMPTS

Direct model instruction

🔑 **Example:** "Translate to French:"

➕ interpretable

➖ not optimal

Prompt Tuning

📖 *Lester, B. et al. 2021. The Power of Scale for Parameter-Efficient Prompt Tuning. Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing.*

Learn continuous "soft prompt" embeddings (20-100 "soft" tokens) prepended to the input

- Add learnable embeddings to input: $[e_{\text{soft}}; e_{\text{input}}]$
- Only affects first layer
- Model processes soft prompt like regular tokens

INSPIRATION: HARD PROMPTS

Direct model instruction

🔑 **Example:** "Translate to French:"

➕ interpretable

➖ not optimal

Prompt Tuning

📖 Lester, B. et al. 2021. *The Power of Scale for Parameter-Efficient Prompt Tuning*. *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*.

Learn continuous "soft prompt" embeddings (20-100 "soft" tokens) prepended to the input

- Add learnable embeddings to input: $[e_{\text{soft}}; e_{\text{input}}]$
- Only affects first layer
- Model processes soft prompt like regular tokens

➕ Can match full finetuning on many tasks (with large models)

➖ More effective as models grow (>10B params)

Prefix Tuning

Li, X.L. and Liang, P. 2021. *Prefix-Tuning: Optimizing Continuous Prompts for Generation*.

Prepend v learnable "virtual tokens" to each layer's key and value projections

⚡ $\mathbf{x}_{1:v}^{(\ell)}$ are **parameters of the model**

in the paper, the authors use an MLP to predict the prefix (stabilizes the training)

Prefix Tuning

Li, X.L. and Liang, P. 2021. *Prefix-Tuning: Optimizing Continuous Prompts for Generation*.

Prepend v learnable "virtual tokens" to each layer's key and value projections

$\mathbf{x}_{1:v}^{(\ell)}$ are **parameters of the model**

in the paper, the authors use an MLP to predict the prefix (stabilizes the training)

⊕ More expressive than prompt tuning (affects all layers)

⊖ Can be sensitive to initialization

COMPARISON

BY PARAMETER EFFICIENCY (FEWEST TO MOST)

1. Prompt Tuning: ~0.01% (input embeddings only)
2. IA³: ~0.01% (scaling vectors)
3. Prefix Tuning: ~0.1% (prefix vectors per layer)
4. LoRA: ~0.1-1% (low-rank matrices)
5. Adapters: ~1-5% (bottleneck layers)

COMPARISON

BY PARAMETER EFFICIENCY (FEWEST TO MOST)

1. Prompt Tuning: ~0.01% (input embeddings only)
2. IA³: ~0.01% (scaling vectors)
3. Prefix Tuning: ~0.1% (prefix vectors per layer)
4. LoRA: ~0.1-1% (low-rank matrices)
5. Adapters: ~1-5% (bottleneck layers)

PRACTICAL RECOMMENDATIONS

- LoRA: best default choice (balance of efficiency & performance)
- Prompt/Prefix: very large models (>10B), trained with instruction tuning
- IA³: extreme parameter constraints (very small datasets)

INFERENCE OPTIMIZATION

- **Problem**
 - LLMs are huge: billions to trillions of parameters
 - Inference is slow and memory-intensive
 - Deployment costs are prohibitive

- **Problem**
 - LLMs are huge: billions to trillions of parameters
 - Inference is slow and memory-intensive
 - Deployment costs are prohibitive
- **Optimization goals**
 - Reduce memory footprint
 - Increase throughput (tokens/second)
 - Minimize latency
 - Maintain accuracy

INFERENCE OPTIMIZATION

QUANTIZATION

Quantization

📖 A Survey of Quantization Methods for Efficient Neural Network Inference: 2021. <http://arxiv.org/abs/2103.13630>.

Reduce numerical precision of weights and activations (e.g., FP32 → INT8 → INT4)

- ⊕ 4× smaller memory (FP32 → INT8), 8× (FP32 → INT4)
- ⊕ Faster operations on  specialized hardware
- ⊕ Lower power consumption
- ⊖ Lower precision

Quantization

📖 A Survey of Quantization Methods for Efficient Neural Network Inference: 2021. <http://arxiv.org/abs/2103.13630>.

Reduce numerical precision of weights and activations (e.g., FP32 → INT8 → INT4)

- ➕ 4× smaller memory (FP32 → INT8), 8× (FP32 → INT4)
- ➕ Faster operations on  specialized hardware
- ➕ Lower power consumption
- ➖ Lower precision

KEY DISTINCTIONS

- Weight-only vs. weight+activation quantization
- From *Post-Training Quantization (PTQ)* to *Quantization-Aware Training (QAT)*
- Symmetric vs. asymmetric quantization
- Per-tensor vs. per-channel quantization

CHALLENGES

- Outliers: few extreme values dominate scale
- Per-layer sensitivity: some layers more sensitive to quantization
- Accumulation errors: errors compound across layers

BASIC (DE)QUANTIZATION FORMULA

- Map floating-point to integer: $x_q = \text{round}(x/s) + z$
with a scale factor: $s = \frac{x_{\max} - x_{\min}}{q_{\max} - q_{\min}}$ and z (0 when symmetric)
- Dequantization: $\hat{x} = (x_q - z) \times s$

MULTIPLICATION

We can compute in the quantized space

$$\hat{x} \times \hat{y} = s \times s' (x_q - z) \times (y_q - z')$$

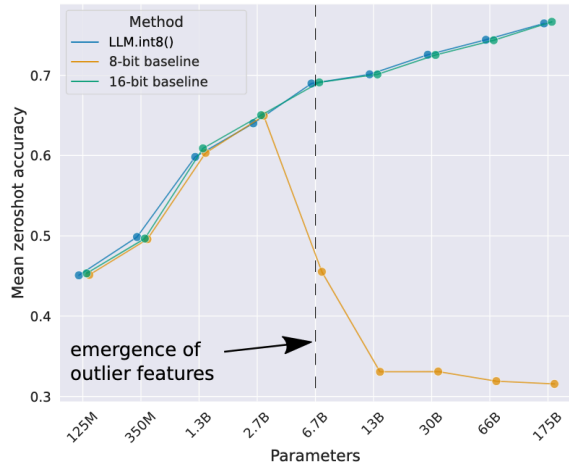
☞ For matrix multiplication $\mathbf{Z} = \mathbf{XY}$, we often use row and column specific quantizer parameters

$$\mathbf{Z}_{ij} = s_i s_j \sum_k (q(\mathbf{X}_{ik}) - z_i) \times (q(\mathbf{Y}_{kj}) - z_j)$$

INFERENCE OPTIMIZATION ► LLM.int8()

LLM.int8(): 8-Bit Matrix Multiplication for Transformers at Scale: 2022. <http://arxiv.org/abs/2208.07339>.

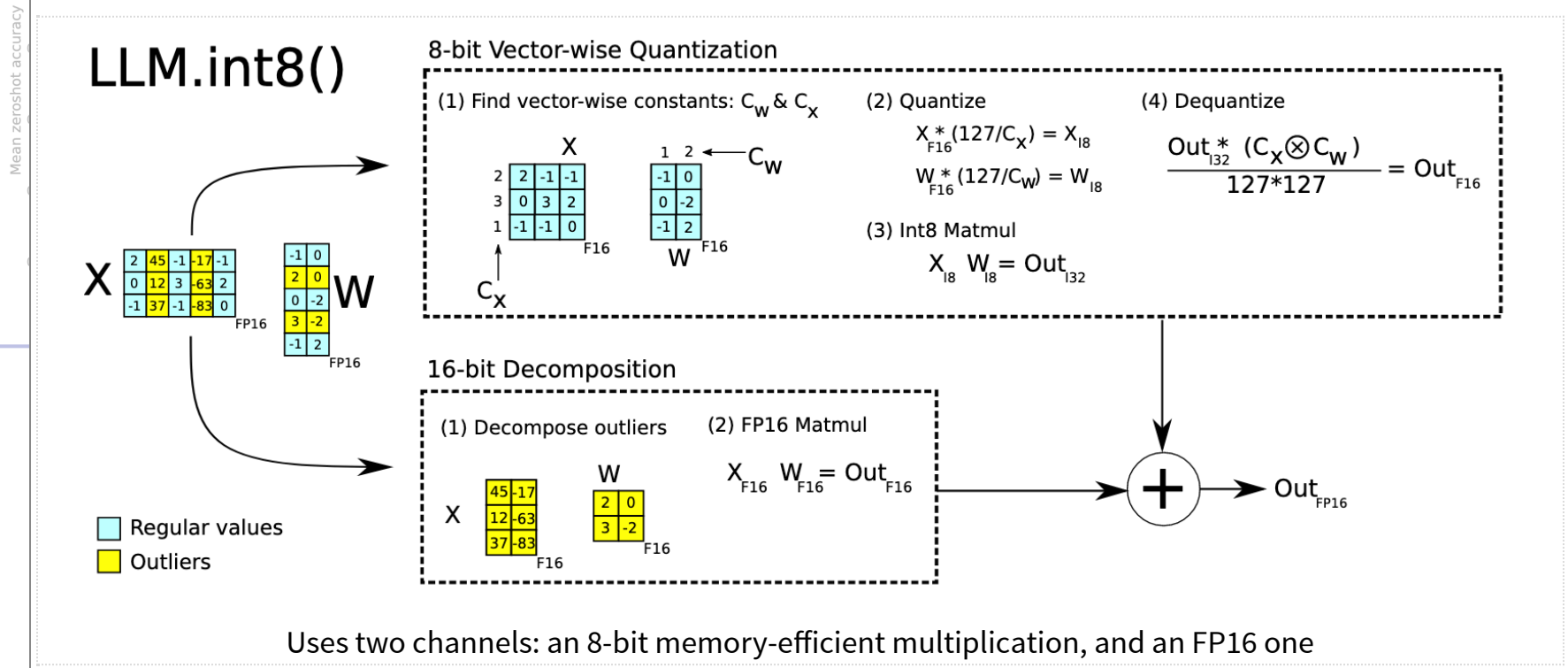
💡 Solves the problem of outliers in LLMs



- 🔧 higher quantization precision at scales beyond 1B parameters
- 🔧 explicitly represent the sparse but systematic large magnitude outlier features that ruin quantization precision once they emerge in all transformer layers starting at scales of 6.7B parameters.

Solves the problem of outliers in LLMs

MORE IN DETAILS



INFERENCE OPTIMIZATION ► ADVANCED QUANTIZATION: GPTQ

📖 Frantar, E. et al. 2022. GPTQ: Accurate Post-Training Quantization for Generative Pre-trained Transformers.

- Layer-wise quantization using second-order information
- Minimize $\|\mathbf{W}\mathbf{X} - \hat{\mathbf{W}}\mathbf{X}\|^2$ where $\mathbf{X}$ is calibration data
- Optimal Brain Quantization: greedily (group of column-wise) quantize weights
- Achieves 4-bit with minimal degradation

INFERENCE OPTIMIZATION ► ADVANCED QUANTIZATION: GPTQ

📖 Frantar, E. et al. 2022. GPTQ: Accurate Post-Training Quantization for Generative Pre-trained Transformers.

- Layer-wise quantization using second-order information
- Minimize $\|WX - \hat{W}X\|^2$ where X is calibration data
- Optimal Brain Quantization: greedily (group of column-wise) quantize weights
- Achieves 4-bit with minimal degradation

Perplexity

Method	Bits	OPT-175B				BLOOM-176B			
		Wiki2	PTB	C4	LAMB. ↑	Wiki2	PTB	C4	LAMB. ↑
Baseline	16	8.34	12.01	10.13	75.59	8.11	14.59	11.71	67.40
RTN	4	10.54	14.22	11.61	71.34	8.37	15.00	12.04	66.70
GPTQ	4	8.37	12.26	10.28	76.80	8.21	14.75	11.81	67.71
RTN	3	7.3e3	8.0e3	4.6e3	0	571.	107.	598.	0.17
GPTQ	3	8.68	12.68	10.67	76.19	8.64	15.57	12.27	65.10
GPTQ	3/g1024	8.45	12.48	10.47	77.39	8.35	15.01	11.98	67.47
GPTQ	3/g128	8.45	12.37	10.36	76.42	8.26	14.89	11.85	67.86

Speedup

GPU	FP16	3bit	Speedup	GPU reduction
A6000 – 48GB	589ms	130ms	4.53×	8 → 2
A100 – 80GB	230ms	71ms	3.24×	5 → 1

INFERENCE OPTIMIZATION ► QUANTIZATION-AWARE TRAINING (QAT)

LLM-QAT: Data-Free Quantization Aware Training for Large Language Models: 2023. <http://arxiv.org/abs/2305.17888>.

• Straight-Through Estimator (STE)

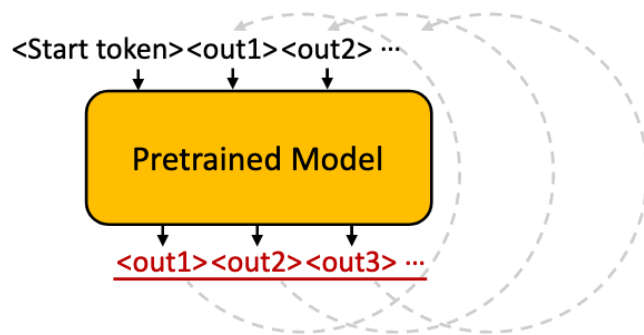
- Forward: quantize

$$\hat{x} = s \cdot \text{clip}(\text{round}(x/s), q_{\min}, q_{\max})$$

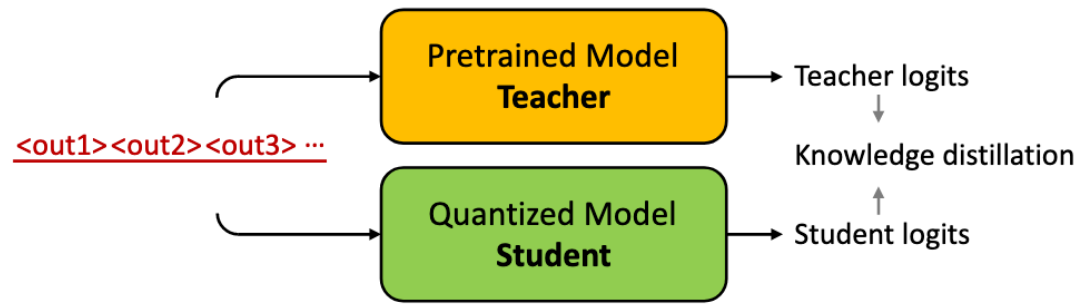
- Backward: bypass quantization $\frac{\partial L}{\partial x} \approx \frac{\partial L}{\partial x_q}$

assumption: the rounding follows a uniform distribution,

$$\hat{x} = x + \epsilon \implies \nabla \mathbb{E}(x + \epsilon) = \nabla x$$



(a) Next token generation



(b) Knowledge distillation for quantized student training

INFERENCE OPTIMIZATION ► QUANTIZATION-AWARE TRAINING (QAT)

LLM-QAT: Data-Free Quantization Aware Training for Large Language Models: 2023. <http://arxiv.org/abs/2305.17888>.

• Straight-Through Estimator (STE)

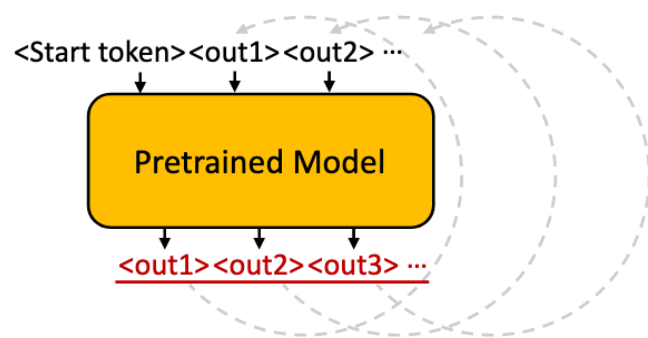
- Forward: quantize

$$\hat{x} = s \cdot \text{clip}(\text{round}(x/s), q_{\min}, q_{\max})$$

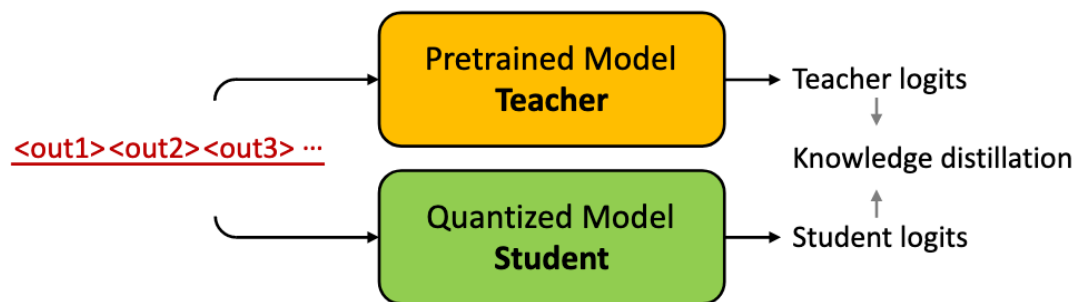
- Backward: bypass quantization $\frac{\partial L}{\partial x} \approx \frac{\partial L}{\partial x_q}$

assumption: the rounding follows a uniform distribution,

$$\hat{x} = x + \epsilon \implies \nabla \mathbb{E}(x + \epsilon) = \nabla x$$



(a) Next token generation



(b) Knowledge distillation for quantized student training

“As a result, we are able to distill the 7B, 13B and 30B LLaMA models with weights and KV cache quantized down to 4-bits.”

INFERENCE OPTIMIZATION

FLASH ATTENTION

⚠ Self-Attention is very costly

Computing the attention logits and the convex combination

⌚ Time complexity $O(d \times n^2)$ (inner product and convex combination)

⚠ Self-Attention is very costly

Computing the attention logits and the convex combination

🔧 Time complexity $O(d \times n^2)$ (inner product and convex combination)

⚠ GPUs are memory-bound

Compute-bound

Time = how fast the processor runs

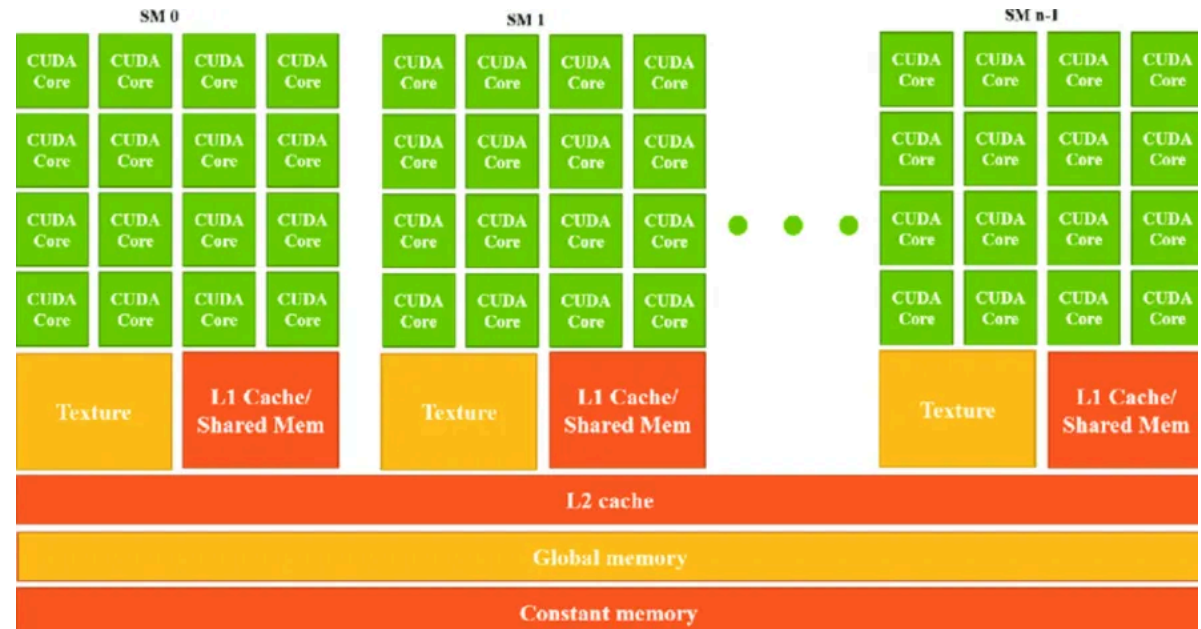
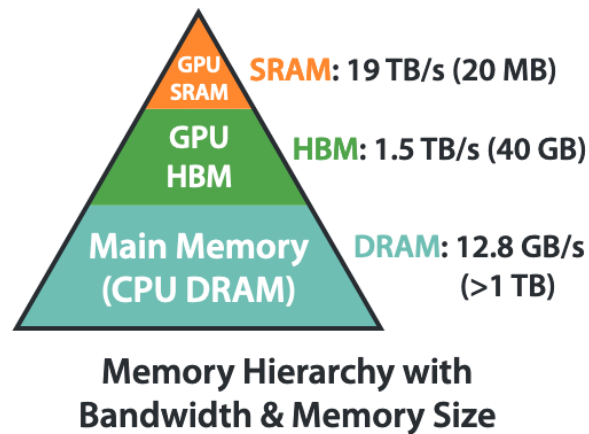
Memory-bound

Time = how fast the memory can feed the processor

🔧 For GPUs, the gap has increased over time. See the [W list of Nvidia graphics processing units](#)

🔧 Memory-intensive operations (ReLU) are memory-bound!

GPU = A SET OF STREAMING MULTIPROCESSOR



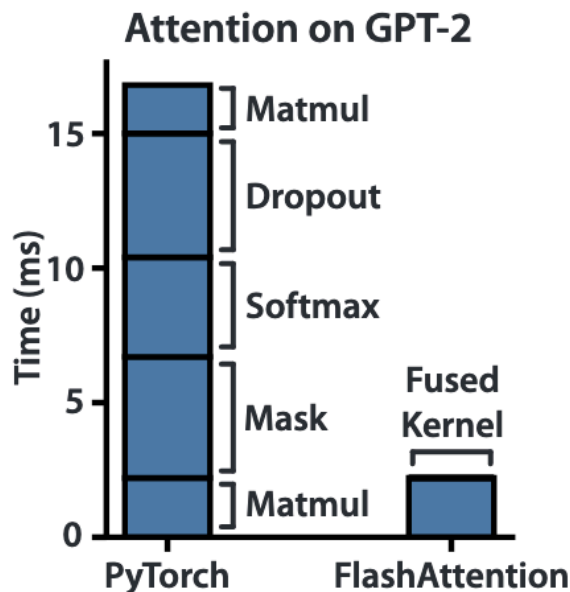
- Each Streaming Multiprocessor is independent and has access to a fast SRAM (128-256 Ko)
- Access to the memory is cached (L2)

- FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness: 2022. <http://arxiv.org/abs/2205.14135>.
- FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning: 2023. <http://arxiv.org/abs/2307.08691>.

💡 Squeeze out most of the GPU processing power

MAIN IDEA = SQUEEZE OUT MOST OF THE GPU PROCESSING POWER

- 🔧 Leverage the bottleneck HBM (High-Bandwidth Memory) and SRAM (on-chip)
- 🔧 Fuse multiple operators in one kernel



Require: Matrices $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{N \times d}$ in HBM.

- 1: Load $\mathbf{Q}, \mathbf{K}$ by blocks from HBM, compute $\mathbf{S} = \mathbf{QK}^\top$, write $\mathbf{S}$ to HBM.
- 2: Read $\mathbf{S}$ from HBM, compute $\mathbf{P} = \text{softmax}(\mathbf{S})$, write $\mathbf{P}$ to HBM.
- 3: Load $\mathbf{P}$ and $\mathbf{V}$ by blocks from HBM, compute $\mathbf{O} = \mathbf{PV}$, write $\mathbf{O}$ to HBM.
- 4: Return $\mathbf{O}$.

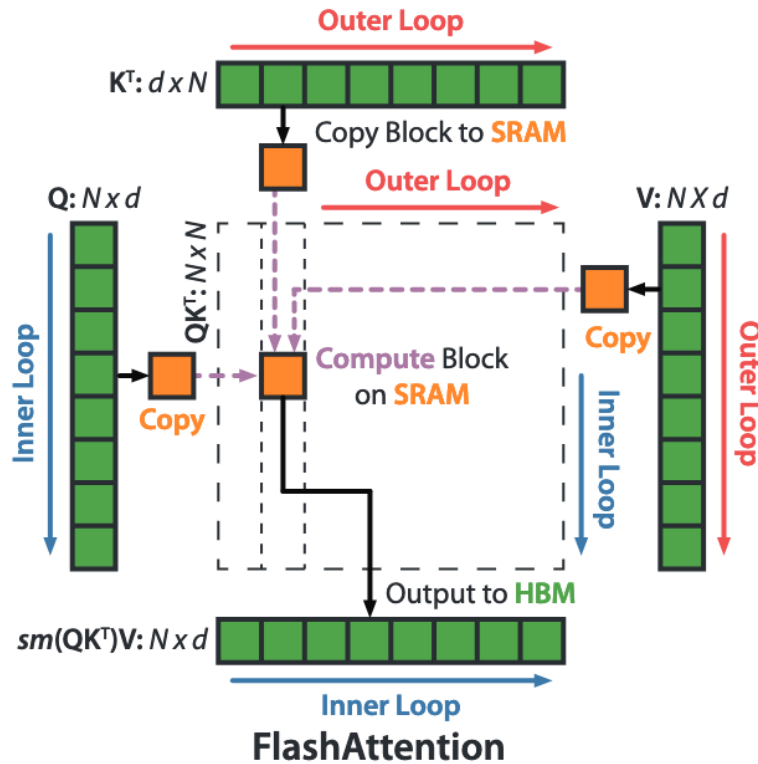
Standard attention algorithm

- Fused kernel: compute attention without materializing full matrix
- Standard attention: $O(n^2)$ memory for attention scores
 - 🔧 Flash Attention: $O(n)$ memory, faster on modern GPUs

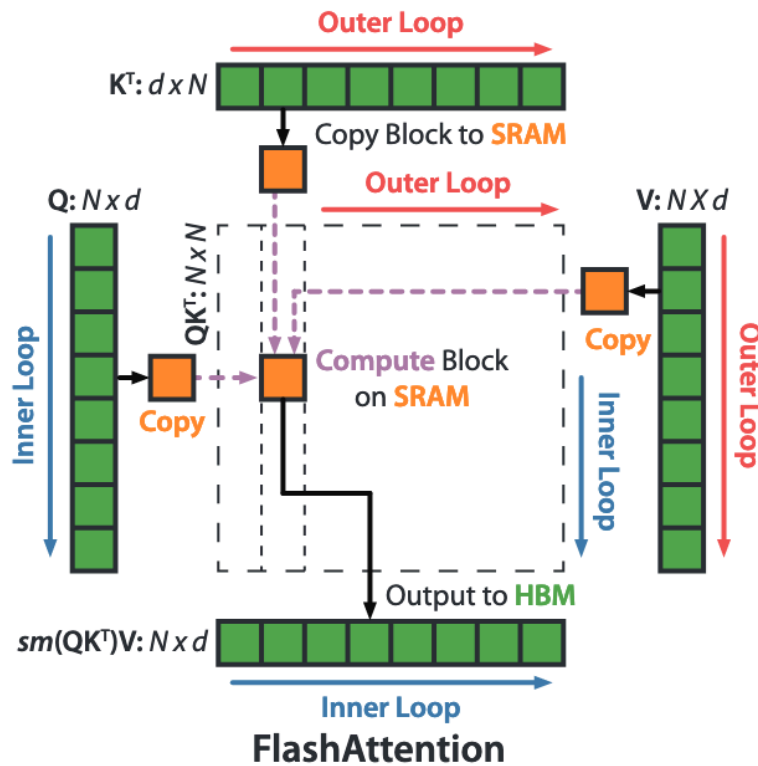
FUSE THE OPERATIONS

We can compute (SRAM)

$$S_{n:p,n':p'} = Q_{n:p,\bullet} K_{\bullet,n:p}^T$$



INFERENCE OPTIMIZATION ► FLASH ATTENTION: LINEAR TRANSFORMATION

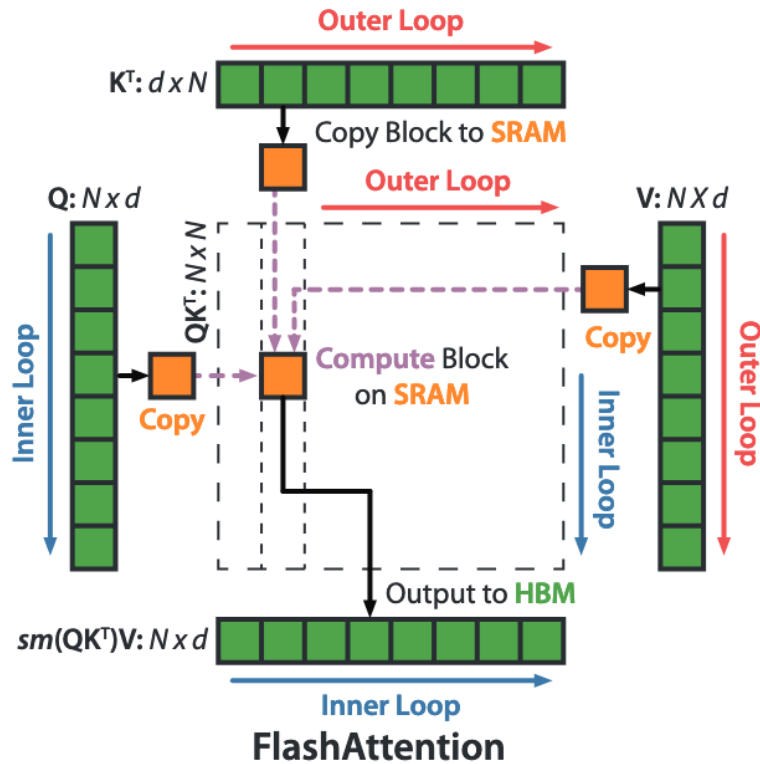


FUSE THE OPERATIONS

We can compute (SRAM)

$$S_{n:p,n':p'} = Q_{n:p,\bullet} K_{\bullet,n:p}^T$$

⚡ But how to compute $sm(QK^T)V$!?



FUSE THE OPERATIONS

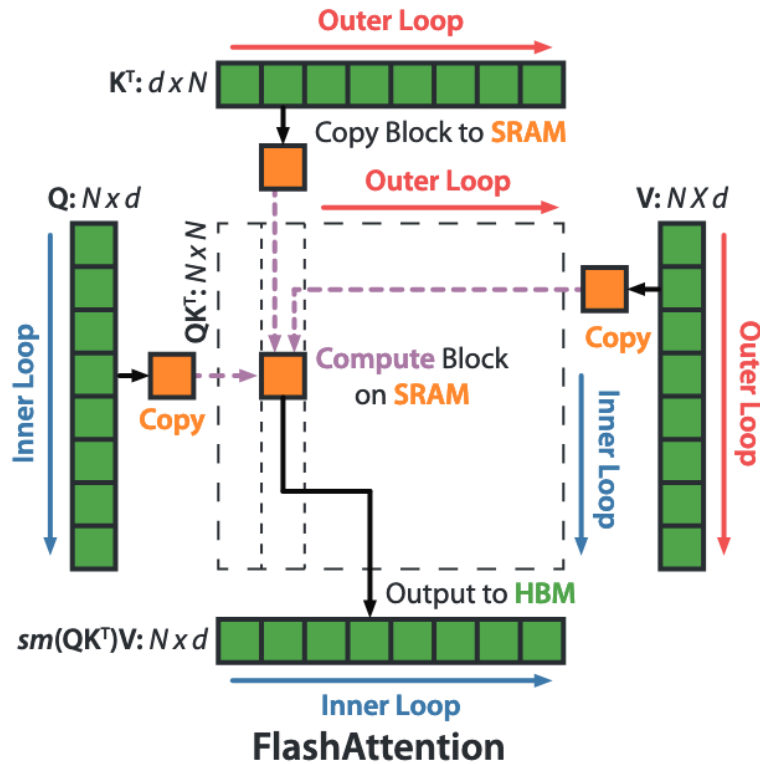
We can compute (SRAM)

$$S_{n:p,n':p'} = Q_{n:p,\bullet} K_{\bullet,n:p}^T$$

⚡ But how to compute $sm(QK^T)V$!?

For one token (over N),

$$\mathbf{o} = \sum_{i=1}^N \frac{\exp(s_i - m)}{\sum_j \exp(s_j - m)} v_i \text{ with } m = \max_j s_j$$



FUSE THE OPERATIONS

We can compute (SRAM)

$$S_{n:p,n':p'} = Q_{n:p,\bullet} K_{\bullet,n:p}^\top$$

⚡ But how to compute $sm(QK^\top)V$!?
For one token (over N),

$$\mathbf{o} = \sum_{i=1}^N \frac{\exp(s_i - m)}{\sum_j \exp(s_j - m)} v_i \text{ with } m = \max_j s_j$$

Posing $m_i = \max_{j \leq i} s_j$, $w_i = \sum_{j \leq i} \exp(s_j - m_i) v_j$, and $l_i = \sum \exp(s_j - m_i)$

$$w_{i+1} = e^{m_i - m_{i+1}} w_i + e^{s_{i+1} - m_{i+1}} v_{i+1} \text{ and } l_{i+1} = e^{m_i - m_{i+1}} l_i + e^{s_{i+1} - m_{i+1}}$$

(iterating over blocks rather than individual positions)

OTHER SMALL THINGS

Attention masks and dropout can be integrated in the algorithm

REDUCING THE MEMORY

🔧 Do not materialize attention (i.e. do not store the score matrix s_{ij})

OVERALL

Huge gains (speed **AND** memory)

Model implementations	OpenWebText (ppl)	Training time (speedup)
GPT-2 small - Huggingface [87]	18.2	9.5 days (1.0×)
GPT-2 small - Megatron-LM [77]	18.2	4.7 days (2.0×)
GPT-2 small - FLASHATTENTION	18.2	2.7 days (3.5×)
GPT-2 medium - Huggingface [87]	14.2	21.0 days (1.0×)
GPT-2 medium - Megatron-LM [77]	14.3	11.5 days (1.8×)
GPT-2 medium - FLASHATTENTION	14.3	6.9 days (3.0×)

INFERENCE OPTIMIZATION

SPECULATIVE DECODING

Speculative Decoding

 *Fast Inference from Transformers via Speculative Decoding: 2023. <http://arxiv.org/abs/2211.17192>.*

Use a small "draft" model to generate candidate tokens, verify with large "target" model

Algorithm

1. Draft model generates k tokens (fast)
2. Target model scores all k tokens
3. Accept tokens that match target model's distribution
4. Resample if mismatch, continue from there

Properties

- Lossless: same output distribution as target model
- Speedup when draft model is accurate enough
- Typical: 2-3× faster with small draft model

```
[START] japan ' s benchmark bond n
[START] japan ' s benchmark nikkei 22 5
[START] japan ' s benchmark nikkei 225 index rose 22 6
[START] japan ' s benchmark nikkei 225 index rose 226 . 69 7 points
[START] japan ' s benchmark nikkei 225 index rose 226 . 69 points , or 0 1
[START] japan ' s benchmark nikkei 225 index rose 226 . 69 points , or 1 . 5 percent , to 10 , 9859
[START] japan ' s benchmark nikkei 225 index rose 226 . 69 points , or 1 . 5 percent , to 10 , 989 . 79 in in
[START] japan ' s benchmark nikkei 225 index rose 226 . 69 points , or 1 . 5 percent , to 10 , 989 . 79 in late morning trading . [END]
```

GPT-like models: draft model = 6M parameters, model = 97M parameters

SPECULATIVE SAMPLING

Let $p(t)$ and $q(t)$ be the two distributions over tokens

We follow these steps (inspired by rejection-sampling)

1. Sample $t \sim q(t)$
2. Accept the sample with a probability $\max(1, \frac{p(t)}{q(t)})$
3. Resample from a probability $p'(t) \propto \max(0, p(t) - q(t))$

☞ This is equivalent to sampling $t \sim p(t)$

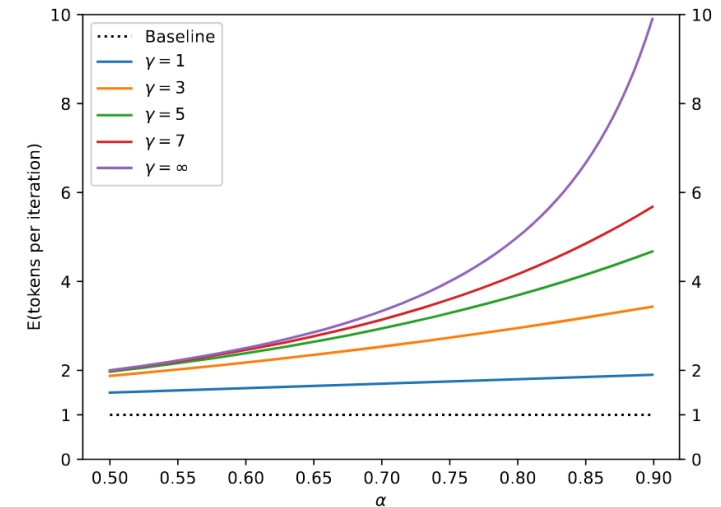
☞ This shows we can sample from q !

EXPECTED NUMBER OF GENERATED TOKENS

$$\mathbb{E}(\# \text{ generated tokens }) = \frac{1 - \alpha^{\gamma+1}}{1 - \alpha}$$

with $\alpha = \mathbb{E}(\beta)$ where β is the probability of accepting a token and γ the length of the sequence sampled from q

$$\alpha = D_{LK}(p, q) = \sum_t \min(p(t), q(t))$$

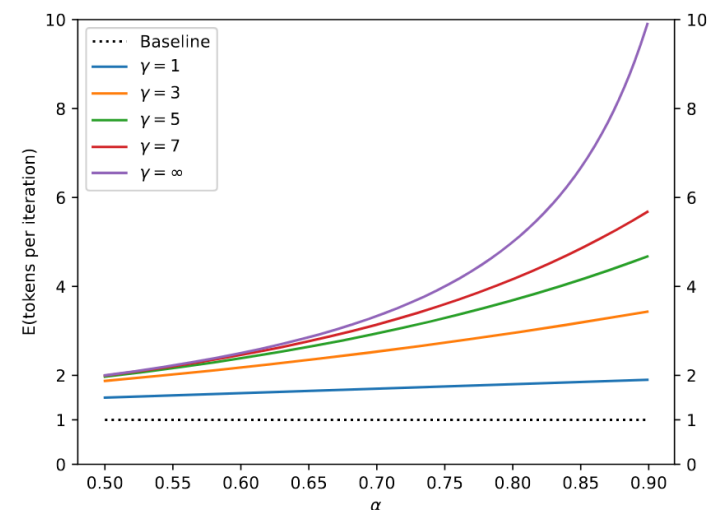


EXPECTED NUMBER OF GENERATED TOKENS

$$\mathbb{E}(\# \text{ generated tokens }) = \frac{1 - \alpha^{\gamma+1}}{1 - \alpha}$$

with $\alpha = \mathbb{E}(\beta)$ where β is the probability of accepting a token and γ the length of the sequence sampled from q

$$\alpha = D_{LK}(p, q) = \sum_t \min(p(t), q(t))$$



EXPECTED IMPROVEMENT (WALL TIME)

With c the ratio of time between inferring p and q

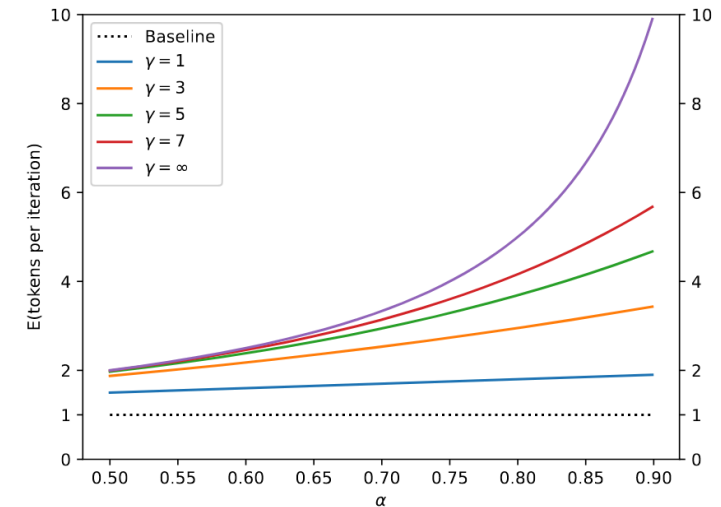
$$\frac{1 - \alpha^{\gamma+1}}{(1 - \alpha)(\gamma c + 1)}$$

EXPECTED NUMBER OF GENERATED TOKENS

$$\mathbb{E}(\# \text{ generated tokens }) = \frac{1 - \alpha^{\gamma+1}}{1 - \alpha}$$

with $\alpha = \mathbb{E}(\beta)$ where β is the probability of accepting a token and γ the length of the sequence sampled from q

$$\alpha = D_{LK}(p, q) = \sum_t \min(p(t), q(t))$$



EXPECTED IMPROVEMENT (WALL TIME)

With c the ratio of time between inferring p and q

$$\frac{1 - \alpha^{\gamma+1}}{(1 - \alpha)(\gamma c + 1)}$$

α	γ	OPERATIONS	SPEED
0.6	2	1.53X	1.96X
0.7	3	1.58X	2.53X
0.8	2	1.23X	2.44X
0.8	5	1.63X	3.69X
0.9	2	1.11X	2.71X
0.9	10	1.60X	6.86X

Theoretical memory-computation trade-off

EMPIRICAL RESULTS

TASK	M_q	TEMP	γ	α	SPEED
ENDE	T5-SMALL ★	0	7	0.75	3.4X
ENDE	T5-BASE	0	7	0.8	2.8X
ENDE	T5-LARGE	0	7	0.82	1.7X
ENDE	T5-SMALL ★	1	7	0.62	2.6X
ENDE	T5-BASE	1	5	0.68	2.4X
ENDE	T5-LARGE	1	3	0.71	1.4X
CNNDM	T5-SMALL ★	0	5	0.65	3.1X
CNNDM	T5-BASE	0	5	0.73	3.0X
CNNDM	T5-LARGE	0	3	0.74	2.2X
CNNDM	T5-SMALL ★	1	5	0.53	2.3X
CNNDM	T5-BASE	1	3	0.55	2.2X
CNNDM	T5-LARGE	1	3	0.56	1.7X

Empirical results for speeding up inference from a T5XXL 11B model

CONCLUSION

- ✚ Wall-time computation improves
- ✖ Computation cost increases
- ✖ Needs a good matching pair of models

EFFICIENT & SCALABLE ARCHITECTURES

- Challenges with scaling
 - Quadratic attention complexity: $O(n^2)$
 - Memory wall: parameters don't fit in GPU
 - Communication overhead in distributed training

- **Challenges with scaling**
 - Quadratic attention complexity: $O(n^2)$
 - Memory wall: parameters don't fit in GPU
 - Communication overhead in distributed training
- **Solutions**
 - Sparse architectures (MoE)
 - Efficient attention patterns
 - Compression techniques

EFFICIENT & SCALABLE ARCHITECTURES

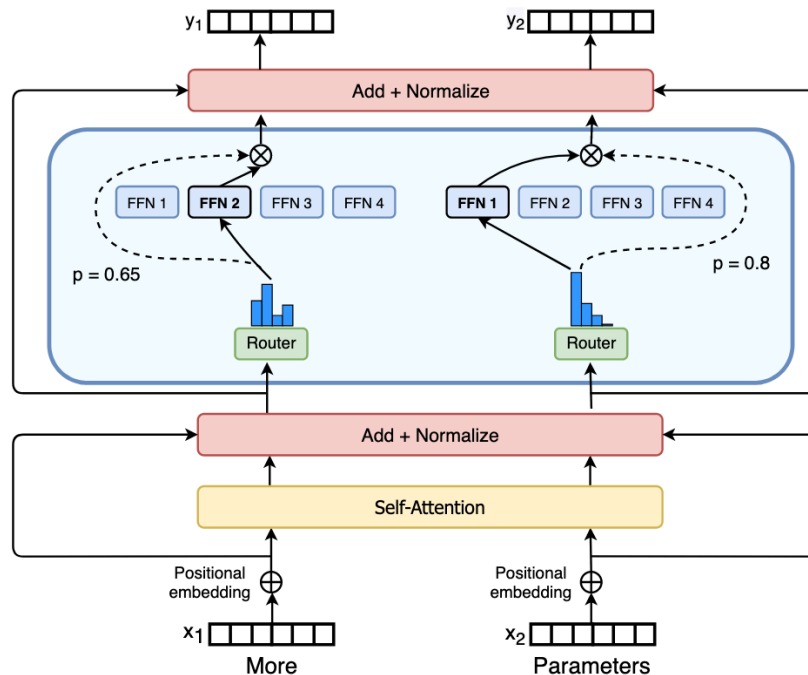
MIXTURE OF EXPERTS

EFFICIENT & SCALABLE ARCHITECTURES ► MIXTURE OF EXPERTS (MOE)

📖 *Switch Transformers: Scaling to Trillion Parameter Models with Simple and Efficient Sparsity: 2022.* <http://arxiv.org/abs/2101.03961>.

► Mixture of Experts

Replace dense FFN layers with multiple "expert" networks, route each token to subset of experts



• Architecture

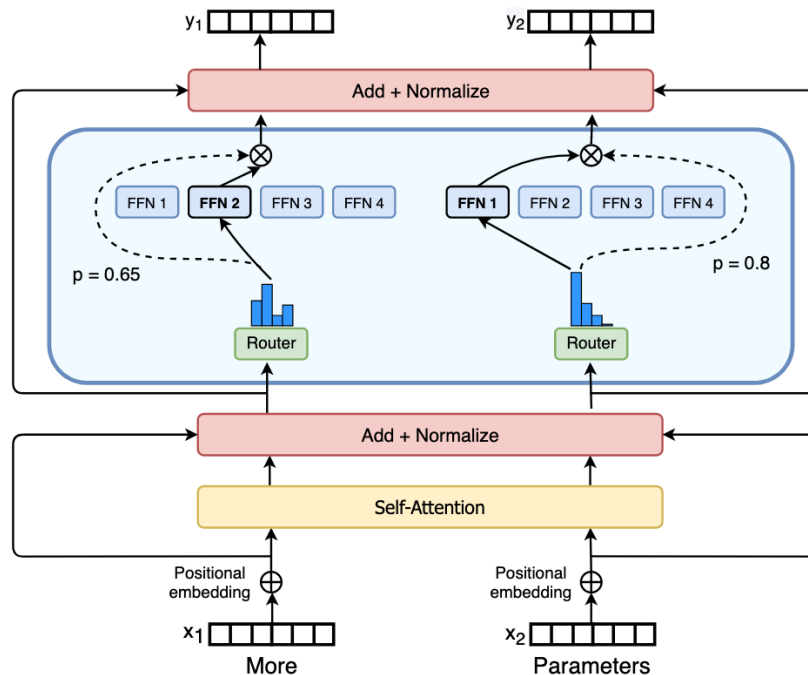
- Router network: learns to assign tokens to experts
- k experts selected per token (e.g., $k = 2$ out of 128)
- Only activated experts compute gradients

EFFICIENT & SCALABLE ARCHITECTURES ► MIXTURE OF EXPERTS (MOE)

📖 *Switch Transformers: Scaling to Trillion Parameter Models with Simple and Efficient Sparsity: 2022. <http://arxiv.org/abs/2101.03961>.*

Mixture of Experts

Replace dense FFN layers with multiple "expert" networks, route each token to subset of experts



Architecture

- Router network: learns to assign tokens to experts
- k experts selected per token (e.g., $k = 2$ out of 128)
- Only activated experts compute gradients

Sparse routing

- Top- k routing: select k experts with highest scores
- Gating function:

$$y = \sum_{i \in \text{top-}k} g_i(x) E_i(x)$$

- Expert capacity: load balancing to prevent overload

WAIT... WHY THIS DOES IMPROVE THINGS?

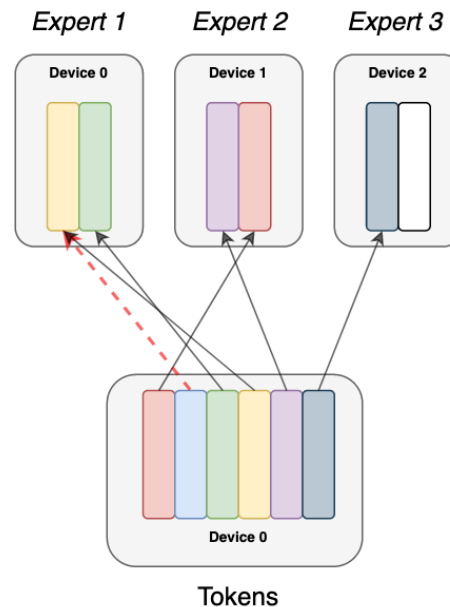
WAIT... WHY THIS DOES IMPROVE THINGS?

- Each expert is assigned to a device...
- and has a bounded capacity

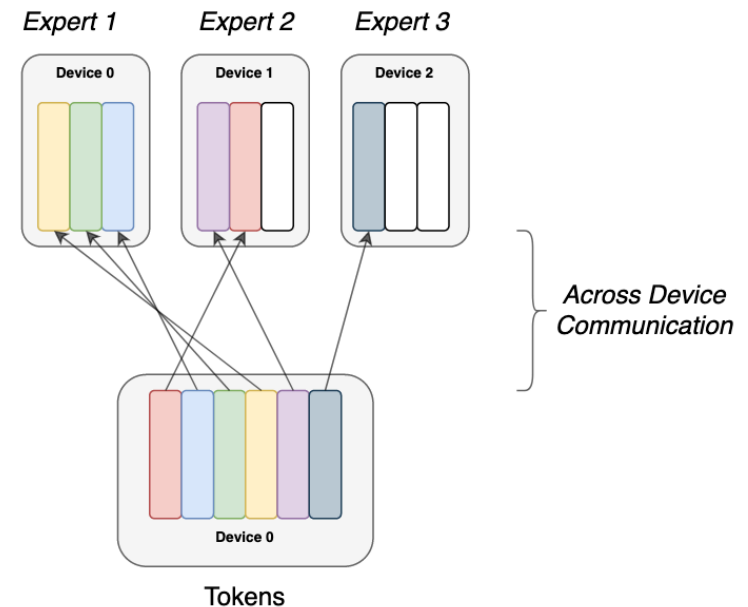
Terminology

- Experts:** Split across devices, each having their own unique parameters. Perform standard feed-forward computation.
- Expert Capacity:** Batch size of each expert. Calculated as $(\text{tokens_per_batch} / \text{num_experts}) * \text{capacity_factor}$
- Capacity Factor:** Used when calculating expert capacity. Expert capacity allows more buffer to help mitigate token overflow during routing.

(Capacity Factor: 1.0)



(Capacity Factor: 1.5)



TRAINING CHALLENGES

⚠ **Load imbalancing:** some experts overused, others ignored

🔧 Add auxiliary loss to encourage balanced routing:

$\sum_{i=1}^N f_i \cdot \sum_j P(i|j)$ with f_i the fraction of tokens dispatched to i and $P(i|j)$ the probability that the j th token is assigned to expert i

⚠ **Expert collapse:** experts become redundant

📖 *Chi, Z. et al. 2022. On the Representation Collapse of Sparse Mixture of Experts.*

🔧 Use a much lower-dimensional vector space for experts

🔧 Use cosine rather than inner product (to compute the expert-token match)

BENEFITS

- Sparse computation: only small fraction of parameters active
- Scale to trillions of parameters (e.g., Switch Transformer)
- Better quality per FLOP than dense models

EFFICIENT & SCALABLE ARCHITECTURES

LONG-RANGE TRANSFORMERS

⚠ Goal

Handle longer sequences efficiently (beyond 2K-8K tokens)

The $O(n^2)$ problem

- Standard attention: every token attends to all others
- Memory: $O(n^2)$, Computation: $O(n^2d)$

⚠ Goal

Handle longer sequences efficiently (beyond 2K-8K tokens)

The $O(n^2)$ problem

- Standard attention: every token attends to all others
- Memory: $O(n^2)$, Computation: $O(n^2d)$

SOLUTIONS

- Linear / Kernel Approximations
- Compressing attention heads
- Reducing the input length
- Hierarchical Attention

► KV Cache

Store computed key and value vectors from previous tokens to avoid recomputation

▼ KV Cache

Store computed key and value vectors from previous tokens to avoid recomputation

- **Mechanics**

- During generation, each token attends to all previous tokens
- Recomputing K, V for all past tokens is wasteful
- Cache stores: $K_{1:t}^{(\ell)} = K^{(\ell)} \mathbf{x}_{1:t}^{(\ell)}$, $V_{1:t} = V^{(\ell)} \mathbf{x}_{1:t}^{(\ell)}$ for each layer

◀ KV Cache

Store computed key and value vectors from previous tokens to avoid recomputation

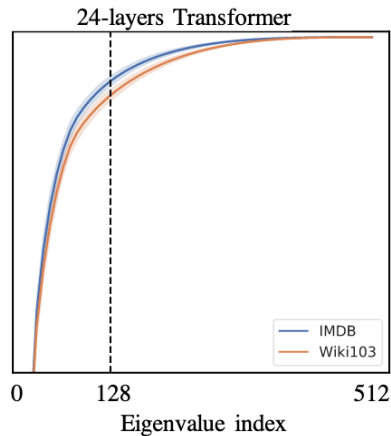
- **Mechanics**

- During generation, each token attends to all previous tokens
- Recomputing K, V for all past tokens is wasteful
- Cache stores: $K_{1:t}^{(\ell)} = K^{(\ell)} \mathbf{x}_{1:t}^{(\ell)}$, $V_{1:t} = V^{(\ell)} \mathbf{x}_{1:t}^{(\ell)}$ for each layer

- **Memory requirements**

- Per token: $2 \times n_{\text{layers}} \times d_{\text{model}}$ (for K and V)
- For 1000 tokens, 40 layers, 4096 dim: ~1.3GB (FP16)
- Dominates memory for long sequences

📖 Wang, S. et al. 2020. Linformer: Self-Attention with Linear Complexity.

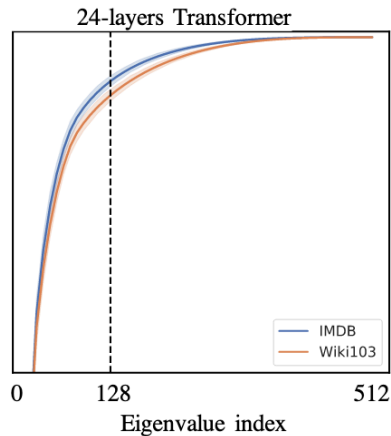


☞ The self-attention matrix P is low rank (k):

$$P \approx P_{\text{low}} = \sum_{i=1}^k \sigma_i u_i v_i^T = \underbrace{[u_1, \dots, u_k]}_{\text{matrix}} \text{diag} \{ \sigma_1, \dots, \sigma_k \} \begin{bmatrix} v_1 \\ \vdots \\ v_k \end{bmatrix} \}_{k}$$

☞ Relies on the W Johnson–Lindenstrauss lemma:

📖 Wang, S. et al. 2020. Linformer: Self-Attention with Linear Complexity.



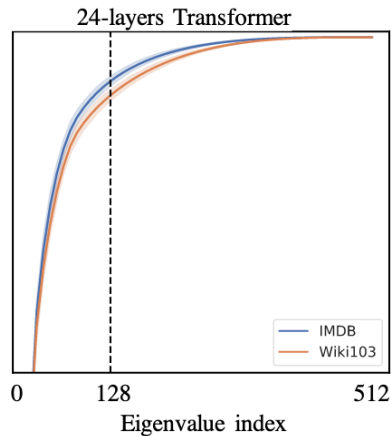
☞ The self-attention matrix P is low rank (k):

$$P \approx P_{\text{low}} = \sum_{i=1}^k \sigma_i u_i v_i^T = \underbrace{[u_1, \dots, u_k]}_{\text{matrix}} \text{diag} \{ \sigma_1, \dots, \sigma_k \} \begin{bmatrix} v_1 \\ \vdots \\ v_k \end{bmatrix} \}^k$$

☞ Relies on the W Johnson–Lindenstrauss lemma:

But this is even more costly !?

Wang, S. et al. 2020. Linformer: Self-Attention with Linear Complexity.



☞ The self-attention matrix P is low rank (k):

$$P \approx P_{\text{low}} = \sum_{i=1}^k \sigma_i u_i v_i^T = \underbrace{[u_1, \dots, u_k]}_{k \times d} \text{diag} \{\sigma_1, \dots, \sigma_k\} \begin{bmatrix} v_1 \\ \vdots \\ v_k \end{bmatrix}_{k \times d}$$

☞ Relies on the Johnson–Lindenstrauss lemma:

But this is even more costly !?

PROPOSED SOLUTION: PROJECT $\mathbb{R}^{N \times d}$ INTO A $\mathbb{R}^{k \times d}$

$$\begin{aligned} \overline{\text{head}_i} &= \text{Attention} \left(QW_i^Q, E_iKW_i^K, F_iVW_i^V \right) \\ &= \underbrace{\text{softmax} \left(\frac{QW_i^Q (E_iKW_i^K)^T}{\sqrt{d_k}} \right)}_{\bar{P}: n \times k} \cdot \underbrace{F_iVW_i^V}_{k \times d} \end{aligned}$$

📖 Choromanski, K. et al. 2020. Rethinking Attention with Performers.

Based on kernel approximations, i.e. $\exp(x \cdot y) = \mathbb{E}_{\omega \sim \mathcal{N}(0; I_d)}(\phi_\omega(x) \cdot \phi_\omega(y))$ with

$$\phi_\omega(x) = \exp\left(-\frac{\|x\|^2}{2}\right) \exp(\omega^\top x)$$

Hence the softmax can be approximated by

$$\frac{\mathbb{E}_{\omega \sim \mathcal{N}(0; I_d)} \left(\sum_j \phi_\omega(q)^\top \phi_\omega(k_j) v_j \right)}{\mathbb{E}_{\omega \sim \mathcal{N}(0; I_d)} \left(\sum_j \phi_\omega(q)^\top \phi_\omega(k_j) \right)}$$

🔧 When using m ($\approx$) samples (ω), the Performer self-attention becomes $O(Nmd)$ – instead of $O(N^2d)$

🔧 More tricks in the paper (e.g. how to better sample ω)

TRANSFORMERS DON'T JUST NEED FAST ATTENTION — THEY NEED PRECISE ATTENTION

- Softmax gives:
 - Scale invariance
 - Implicit competition between tokens
 - Stable gradients
- Kernel attention often:
 - Loses normalization
 - Requires ad-hoc rescaling
 - Produces unstable attention mass

IN SHORT

- ☞ kernel-based approaches breaks “sharp attention”
- ☞ much better alternatives relying on exact and hierarchical attention

- **Multi-Query Attention (MQA)**

📖 *Fast Transformer Decoding: One Write-Head Is All You Need: 2019. <http://arxiv.org/abs/1911.02150>.*

- Share K, V across all attention heads
- Only Q is multi-headed
- Reduces KV cache size by factor of n_{heads}

- **Multi-Query Attention (MQA)**

📖 *Fast Transformer Decoding: One Write-Head Is All You Need: 2019. <http://arxiv.org/abs/1911.02150>.*

- Share K, V across all attention heads
- Only Q is multi-headed
- Reduces KV cache size by factor of n_{heads}

- **Grouped-Query Attention (GQA)**

📖 *GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints: 2023. <http://arxiv.org/abs/2305.13245>.*

- Compromise: group heads, share K, V within groups
- Example: 32 Q heads, 8 KV heads (4 heads per group)
- Better quality than MQA, still reduces cache

- **Multi-Query Attention (MQA)**

📖 *Fast Transformer Decoding: One Write-Head Is All You Need: 2019.* <http://arxiv.org/abs/1911.02150>.

- Share K, V across all attention heads
- Only Q is multi-headed
- Reduces KV cache size by factor of n_{heads}

- **Grouped-Query Attention (GQA)**

📖 *GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints: 2023.* <http://arxiv.org/abs/2305.13245>.

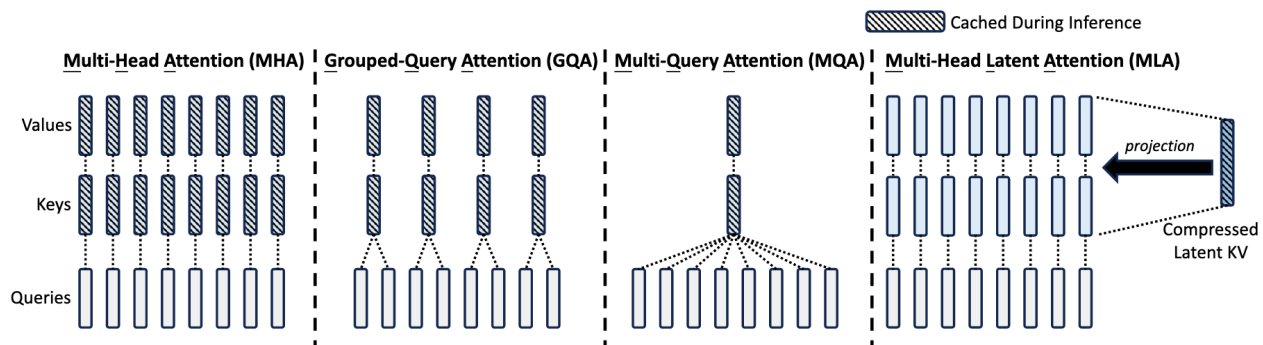
- Compromise: group heads, share K, V within groups
- Example: 32 Q heads, 8 KV heads (4 heads per group)
- Better quality than MQA, still reduces cache

- **Multi-head Latent Attention (MLA)**

📖 *DeepSeek-V2: A Strong, Economical, and Efficient Mixture-of-Experts Language Model: 2024.* <http://arxiv.org/abs/2405.04434>.

- Compress into a lower-dimensional latent space (much less than $2 \times d_h \times n_h$)
- Shared key/value space
- Better quality than GQA

COMPARING THE DIFFERENT ATTENTION MECHANISMS



Source:  *DeepSeek-V2: A Strong, Economical, and Efficient Mixture-of-Experts Language Model: 2024.*

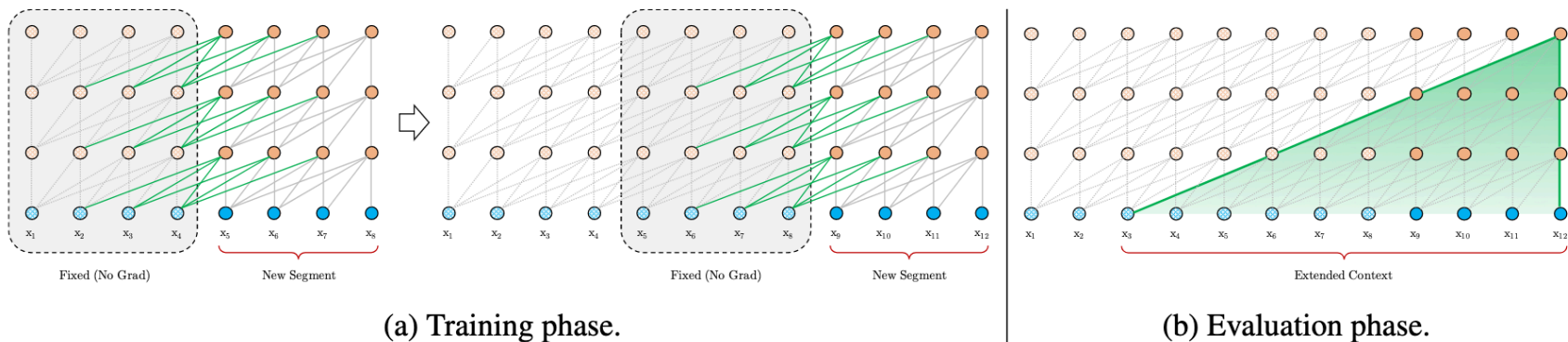
<http://arxiv.org/abs/2405.04434>.

How to increase the perception field in transformers without increasing the training cost (too much)?

How to increase the perception field in transformers without increasing the training cost (too much)?

Transformer-XL

 Dai, Z. et al. 2019. *Transformer-XL: Attentive Language Models Beyond a Fixed-Length Context*.



- ☞ Memory = M previous steps before the context; uses "stop-gradient" to learn
- ☞ Use relative position embeddings to cater for longer contexts

⚠ Transformer-XL still limited

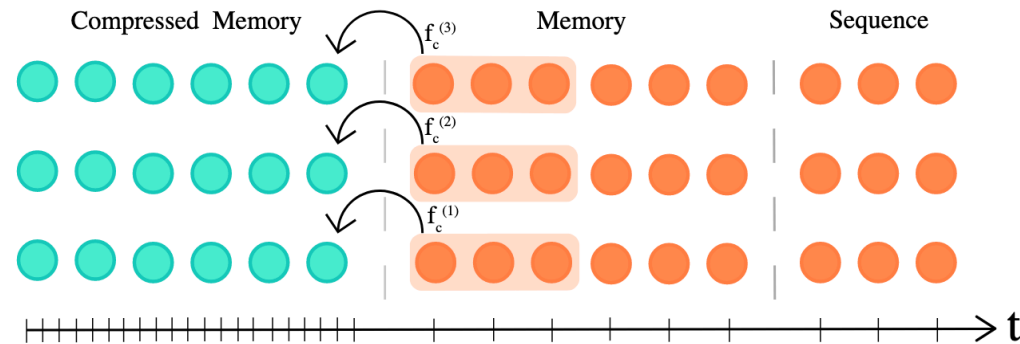
it keeps a fixed-length memory of past segment activations...
but eventually drops old states once memory fills up.

⚠ Transformer-XL still limited

it keeps a fixed-length memory of past segment activations...
but eventually drops old states once memory fills up.

🔧 The Compressive Transformer improves this by maintaining two memory buffers:

📖 Rae, J.W. et al. 2019. *Compressive Transformers for Long-Range Sequence Modelling*.



Different strategies are considered:

Compression function (1) max/mean pooling; (2) 1D (dilated-)convolution; (3) most-used (average attention)

Loss Reconstruction or Attention-based Reconstruction

CONCLUSION

CONCLUSION

- ⊕ Longer range than vanilla transformers
- ⊖ Still limited range
- ⊖ Hard to train

CONCLUSION

- ➕ Longer range than vanilla transformers
- ➖ Still limited range
- ➖ Hard to train

ALTERNATIVE

- 🔑 **sparse**: simpler alternatives that work well
- 🔑 **hierarchical**: allow long span **and** precise information fetching

- **Local (sliding window) attention**
 - Each token attends to fixed-size neighborhood (w tokens)
 - Complexity: $O(n \cdot w)$ where $w \ll n$
 - Captures local context (like CNN)

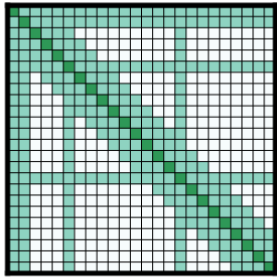
- **Local (sliding window) attention**
 - Each token attends to fixed-size neighborhood (w tokens)
 - Complexity: $O(n \cdot w)$ where $w \ll n$
 - Captures local context (like CNN)
- **Strided (dilated) attention**
 - Attend to every k -th token ($k = 2, 4, 8, \dots$)
 - Provides longer-range connections with gaps
 - Like dilated convolutions

- **Local (sliding window) attention**
 - Each token attends to fixed-size neighborhood (w tokens)
 - Complexity: $O(n \cdot w)$ where $w \ll n$
 - Captures local context (like CNN)
- **Strided (dilated) attention**
 - Attend to every k -th token ($k = 2, 4, 8, \dots$)
 - Provides longer-range connections with gaps
 - Like dilated convolutions
- **Global attention**
 - Few special tokens attend to all positions
 - All tokens attend to these global tokens
 - Example: CLS token, summary tokens

- **Local (sliding window) attention**
 - Each token attends to fixed-size neighborhood (w tokens)
 - Complexity: $O(n \cdot w)$ where $w \ll n$
 - Captures local context (like CNN)
- **Strided (dilated) attention**
 - Attend to every k -th token ($k = 2, 4, 8, \dots$)
 - Provides longer-range connections with gaps
 - Like dilated convolutions
- **Global attention**
 - Few special tokens attend to all positions
 - All tokens attend to these global tokens
 - Example: CLS token, summary tokens
- **Random attention**
 - Random subset of positions per token
 - Provides unexpected long-range connections
 - Helps information flow

Longformer

📖 Beltagy, I. et al. 2020. Longformer: The Long-Document Transformer.



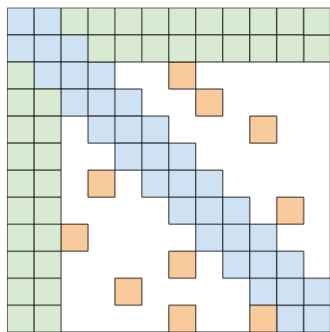
(d) Global+sliding window

RoBERTa-based model that combines local windowed attention with task-specific global attention

- All tokens: local sliding window (window size $w = 512$)
- Selected tokens: global attention (e.g., CLS, question tokens)
- Complexity: $O(n \cdot w)$ instead of $O(n^2)$

BigBird

📖 Zaheer, M. et al. 2020. Big Bird: Transformers for Longer Sequences.



(d) BIGBIRD

Sparse attention combining local, global, and random connections

🔑 Adds random attention to Longformer

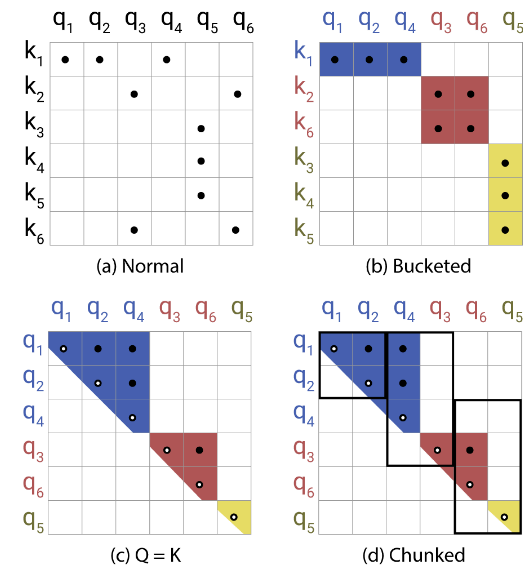
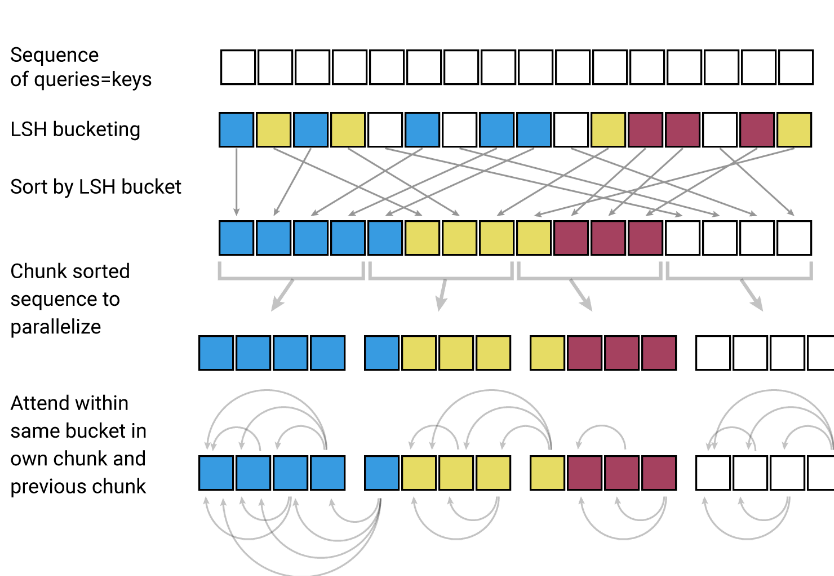
EFFICIENT & SCALABLE ARCHITECTURES ► HIERARCHICAL ATTENTION: REFORMER

📖 Kitaev, N. et al. 2020. Reformer: The Efficient Transformer. ICLR.

KEY IDEA

The problem of finding nearest neighbors quickly in high-dimensional spaces can be solved by **locality-sensitive hashing (LSH)**

🔧 In practice, random projections along ω : $\mathbb{1}(\omega \cdot x \geq 0)$



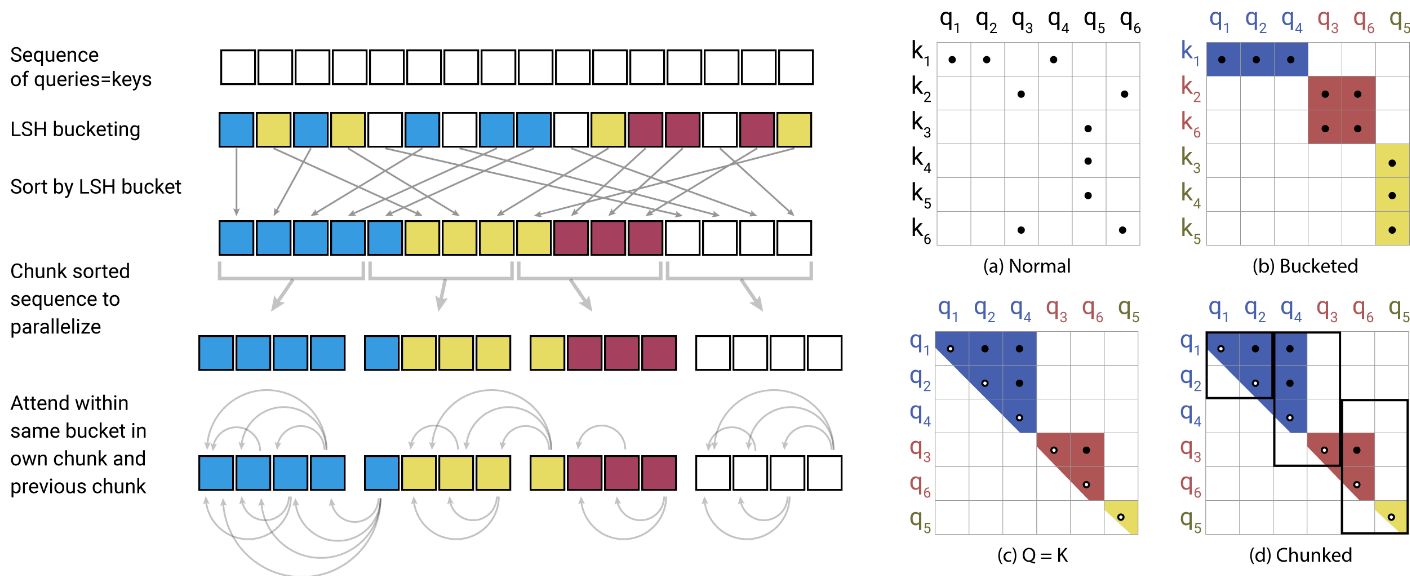
EFFICIENT & SCALABLE ARCHITECTURES ► HIERARCHICAL ATTENTION: REFORMER

📖 Kitaev, N. et al. 2020. Reformer: The Efficient Transformer. ICLR.

KEY IDEA

The problem of finding nearest neighbors quickly in high-dimensional spaces can be solved by **locality-sensitive hashing (LSH)**

🔧 In practice, random projections along ω : $\mathbb{1}(\omega \cdot x \geq 0)$



➕ Allows to handle long spans (depending on the number of buckets)

➖ Having balanced buckets is not easy, bucketing can be harmful (blurs attention – related to Performer problems)

📖 Tay, Y. et al. 2020. Sparse Sinkhorn Attention. Proceedings of the 37th International Conference on Machine Learning, ICML 2020, 13-18 July 2020, Virtual Event.

KEY IDEA

🔧 Replace discrete LSH sorting with a differentiable, soft permutation of queries and keys:

1. Sinkhorn normalization + Gumbel-Softmax to get a permutation matrix approximation
2. Block-level reordering (not token level)

◀ Sinkhorn normalization

🔧 A permutation matrix is a special case of a doubly stochastic matrix (rows/columns sum to 1)

🔧 Sinkhorn Normalization = a doubly stochastic matrix $S(R)$ can be constructed from a non-negative R

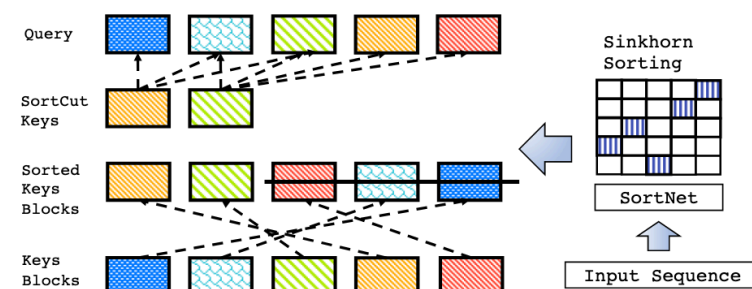
◀ Gumbel Softmax

$$GS(X) = S \left(\frac{X + \epsilon}{\tau} \right)$$

with ϵ a $\mathcal{W}$ Gumbel distribution noise $G(0, 1)$

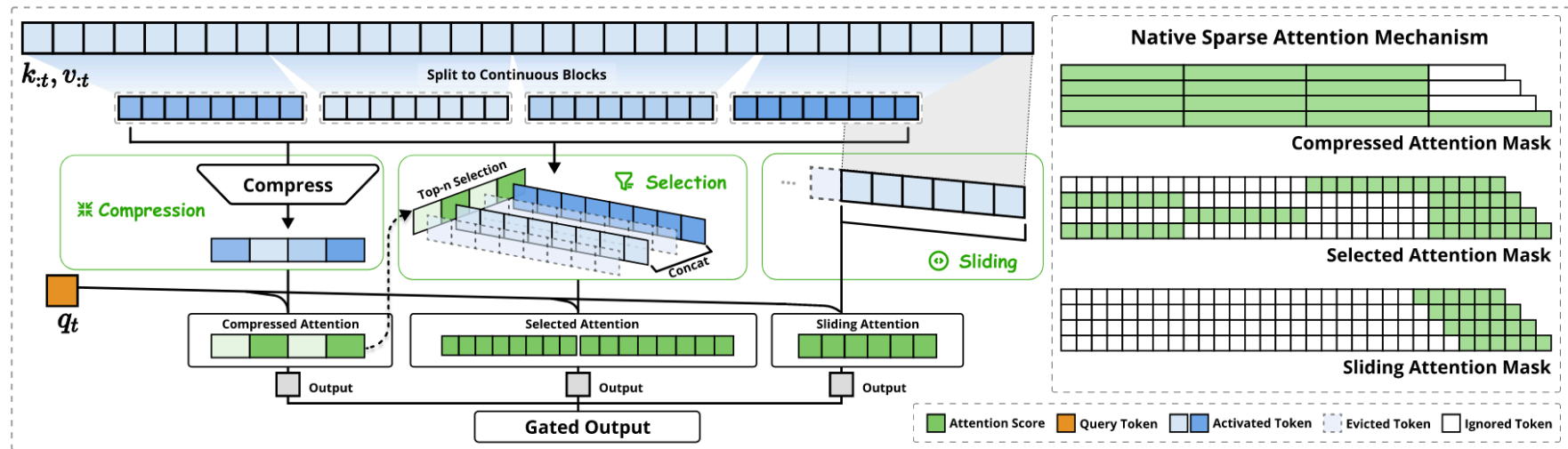
🔧 $GS(X)$ converges to a permutation matrix when $\tau \rightarrow 0$

🔧 We can take the top- K blocks = *SortCut scheme*



EFFICIENT & SCALABLE ARCHITECTURES ► HIERARCHICAL ATTENTION: NATIVE SPARSE ATTENTION

Native Sparse Attention: Hardware-Aligned and Natively Trainable Sparse Attention: 2025. <http://arxiv.org/abs/2502.11089>.

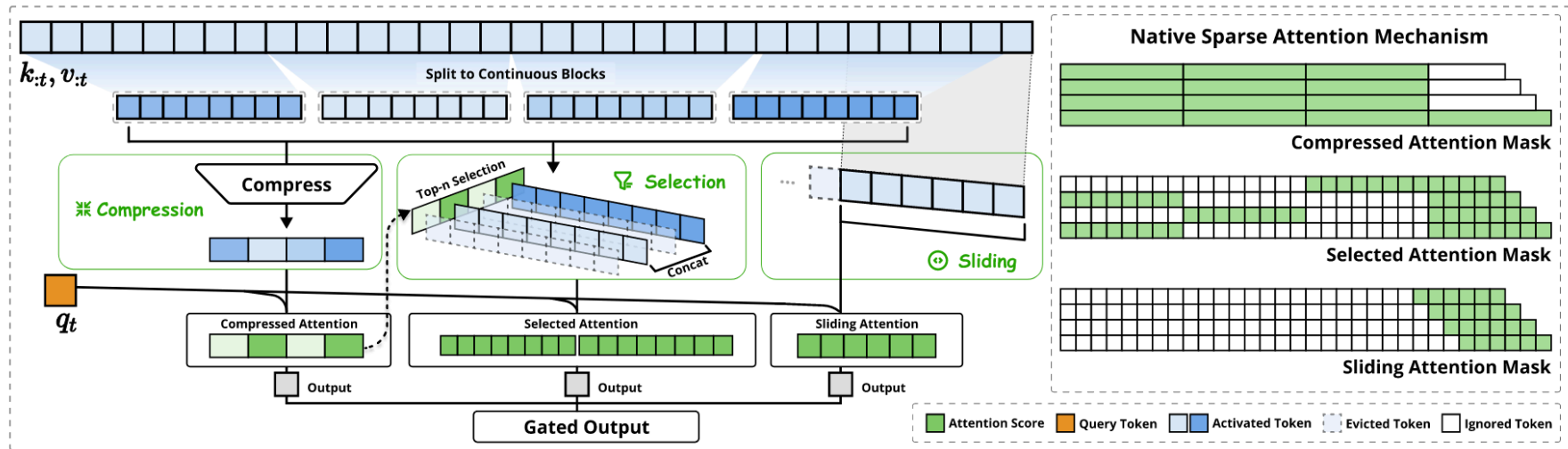


THREE ATTENTION MECHANISMS

1. Sliding attention
2. Block-wise attention (compressed keys and values)
3. Selected attention (using top- k blocks from previous step) – very related to SinkHorn attention

EFFICIENT & SCALABLE ARCHITECTURES ► HIERARCHICAL ATTENTION: NATIVE SPARSE ATTENTION

📖 *Native Sparse Attention: Hardware-Aligned and Natively Trainable Sparse Attention: 2025.* <http://arxiv.org/abs/2502.11089>.

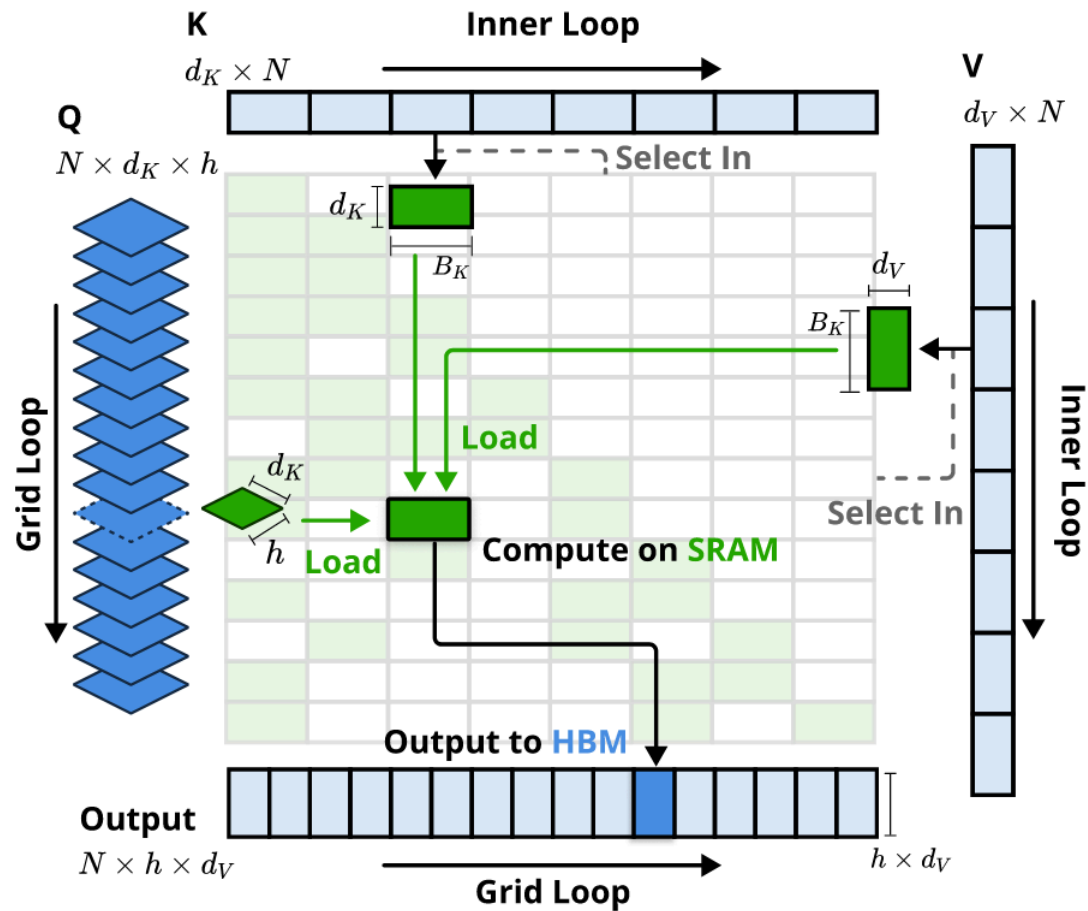


THREE ATTENTION MECHANISMS

1. Sliding attention
2. Block-wise attention (compressed keys and values)
3. Selected attention (using top- k blocks from previous step) – very related to SinkHorn attention

⊕ Combines compressed attention with fine-grained localisation

LEVERAGES FLASH ATTENTION



WHAT DOESN'T WORK WELL

- **Linear/kernel approximations** (Linformer, Performer)
 - Lose attention precision and sharpness
 - Training instability, ad-hoc rescaling needed
 - Not used in production LLMs

WHAT WORKS: HIERARCHICAL + SPARSE + EXACT

- **Multi-level access patterns**
 - Sliding window (local context)
 - Block-wise compression (coarse global view)
 - Selected blocks (fine-grained global attention)
- **Examples:** Native Sparse Attention, Sinkhorn Attention
- **Key insight:** Combine fast approximate selection with exact attention

WHAT WORKS: HIERARCHICAL + SPARSE + EXACT

- **Multi-level access patterns**
 - Sliding window (local context)
 - Block-wise compression (coarse global view)
 - Selected blocks (fine-grained global attention)
- **Examples:** Native Sparse Attention, Sinkhorn Attention
- **Key insight:** Combine fast approximate selection with exact attention

KV CACHE OPTIMIZATION

- **Multi-head Latent Attention (MLA):** compress cached states
- **GQA/MQA:** share key-value heads across queries
- Essential for handling long contexts at inference time

WHAT WORKS: HIERARCHICAL + SPARSE + EXACT

- Multi-level access patterns
 - Sliding window (local context)
 - Block-wise compression (coarse global view)
 - Selected blocks (fine-grained global attention)
- **Examples:** Native Sparse Attention, Sinkhorn Attention
- **Key insight:** Combine fast approximate selection with exact attention

KV CACHE OPTIMIZATION

- Multi-head Latent Attention (MLA): compress cached states
- GQA/MQA: share key-value heads across queries
- Essential for handling long contexts at inference time

🔧 Modern LLMs use **sparse, learnable attention patterns** with **exact computation** where it matters

EFFICIENT & SCALABLE ARCHITECTURES

MODEL COMPRESSION

 *Distilling the Knowledge in a Neural Network: 2015. <http://arxiv.org/abs/1503.02531>.*

WHY DISTILLATION?

- Large models are powerful but resource-intensive
- Distillation transfers knowledge from a large "teacher" model to a smaller "student" model
- Enables deployment on resource-constrained devices

 *Distilling the Knowledge in a Neural Network: 2015. <http://arxiv.org/abs/1503.02531>.*

WHY DISTILLATION?

- Large models are powerful but resource-intensive
- Distillation transfers knowledge from a large "teacher" model to a smaller "student" model
- Enables deployment on resource-constrained devices

HOW TO ALIGN

- Soft labels (probabilities) from the teacher
- Intermediate representations (features, attention maps)
- Task-specific objectives

 Sanh, V. et al. 2019. *DistilBERT, a Distilled Version of BERT: Smaller, Faster, Cheaper and Lighter*.

Teacher

BERT-based (12 layers, 110M parameters)

Student

DistilBERT (6 layers, 66M parameters – 40% smaller) *initialized by taking one layer out of two*

Distillation loss

$$\mathcal{L} = \alpha \mathcal{L}_{MLM} + (1 - \alpha) \mathcal{L}_{KD}$$

where $\mathcal{L}_{MLM}$ is the cross-entropy loss on masked language modeling, and $\mathcal{L}_{KD}$ is the Kullback-Leibler divergence between student and teacher outputs with temperature T :

$$p_i = \frac{\exp(z_i/T)}{\sum_j \exp(z_j/T)}$$

 Sanh, V. et al. 2019. DistilBERT, a Distilled Version of BERT: Smaller, Faster, Cheaper and Lighter.

Teacher

BERT-based (12 layers, 110M parameters)

Student

DistilBERT (6 layers, 66M parameters – 40% smaller) *initialized by taking one layer out of two*

Distillation loss

$$\mathcal{L} = \alpha \mathcal{L}_{MLM} + (1 - \alpha) \mathcal{L}_{KD}$$

where $\mathcal{L}_{MLM}$ is the cross-entropy loss on masked language modeling, and $\mathcal{L}_{KD}$ is the Kullback-Leibler divergence between student and teacher outputs with temperature T :

$$p_i = \frac{\exp(z_i/T)}{\sum_j \exp(z_j/T)}$$

 Retains 97% of BERT's performance on GLUE benchmark

- ☞ A relation r is Q-Q, K-K or V-V (queries, keys or values)
- ☞ Divide the teacher/student space into A parts of dimension $d_{teacher}/A_r$ and $d_{student}/A_r$
- ☞ The keys, values and attention maps are computed, concatenated and then split into A_r parts. Noting $x_{(i,\ell,a)}^{(m)}$ the keys/values/attention maps of token i at layer ℓ , part a for the model m (student or teacher), we can get the similarity distribution of each part

$$p(\dots | i, \ell, a, m) = \text{softmax}_j \left(x_{(j,\ell,a)}^{(m)} \cdot x_{(i,\ell,a)}^{(m)} \right)$$

And then we can compute the KL between teacher and student (for layers ℓ and ℓ' , for a given relation r)

$$\mathcal{L}_r = \frac{1}{N \times A_r} \sum_{a=1}^{A_r} \sum_{i=1}^N KL \left(p(\dots | i, \ell, a, m = teacher), p(\dots | i, \ell', a, m = student) \right)$$

- ☞ A relation r is Q-Q, K-K or V-V (queries, keys or values)
- ☞ Divide the teacher/student space into A parts of dimension $d_{teacher}/A_r$ and $d_{student}/A_r$
- ☞ The keys, values and attention maps are computed, concatenated and then split into A_r parts. Noting $x_{(i,\ell,a)}^{(m)}$ the keys/values/attention maps of token i at layer ℓ , part a for the model m (student or teacher), we can get the similarity distribution of each part

$$p(\dots | i, \ell, a, m) = \text{softmax}_j \left(x_{(j,\ell,a)}^{(m)} \cdot x_{(i,\ell,a)}^{(m)} \right)$$

And then we can compute the KL between teacher and student (for layers ℓ and ℓ' , for a given relation r)

$$\mathcal{L}_r = \frac{1}{N \times A_r} \sum_{a=1}^{A_r} \sum_{i=1}^N KL(p(\dots | i, \ell, a, m = teacher), p(\dots | i, \ell', a, m = student))$$

PERFORMANCE

- Retains over 95% of BERT's performance on GLUE benchmark
- Significantly faster inference times

- **Benefits**
 - Very effective way to train smaller models
 - Used for encoders (DistilBERT) and decoders (DistilGPT-2)
 - Can match or exceed performance of training from scratch

- **Benefits**
 - Very effective way to train smaller models
 - Used for encoders (DistilBERT) and decoders (DistilGPT-2)
 - Can match or exceed performance of training from scratch
- **Challenges**
 - Requires access to teacher model during training
 - Not trivial to train – requires careful tuning
 - Temperature and loss weight balancing critical

- **Benefits**
 - Very effective way to train smaller models
 - Used for encoders (DistilBERT) and decoders (DistilGPT-2)
 - Can match or exceed performance of training from scratch
- **Challenges**
 - Requires access to teacher model during training
 - Not trivial to train – requires careful tuning
 - Temperature and loss weight balancing critical
- **Alternatives**
 - Quantization: simpler, often more effective for deployment
 - Training smaller models from scratch with better data
 - Hybrid approaches: distillation + quantization

CONCLUSION

- **Adaptation**
 - PEFT methods enable efficient task-specific tuning
 - LoRA is the current standard for most applications

- **Adaptation**
 - PEFT methods enable efficient task-specific tuning
 - LoRA is the current standard for most applications
- **Inference optimization**
 - Quantization and KV cache are critical for deployment
 - Speculative decoding offers lossless speedup

CONCLUSION ► KEY TAKEAWAYS

- **Adaptation**
 - PEFT methods enable efficient task-specific tuning
 - LoRA is the current standard for most applications
- **Inference optimization**
 - Quantization and KV cache are critical for deployment
 - Speculative decoding offers lossless speedup
- **Efficient architectures**
 - MoE enables massive scaling with sparse computation
 - Better positional encodings extend context length
 - Attention mechanism have evolved: Flash Attention + sparse and hierarchical attention